

國立陽明交通大學

網路工程研究所

碩士論文

Degree Program of Computer Science

National Yang Ming Chiao Tung University

Master Thesis

HiPert: 針對大型語言模型程式碼場景間接提示注入攻擊之三層語法和語意感知防禦框架

HiPert: A Three-Layer Syntax-Semantics-Aware Defense
Against Indirect Prompt Injection in LLM-based Code
Contexts

研 究 生：鄭博涵 (Cheng, Po Han)

指導教授：林盈達 (Lin, Ying-Dar)

中華民國 一一五年六月

June 2026

HiPert: 針對大型語言模型程式碼場景間接提示注入攻擊之三層語法
和語意感知防禦框架

HiPert: A Three-Layer Syntax-Semantics-Aware Defense Against
Indirect Prompt Injection in LLM-based Code Contexts

研 究 生：鄭博涵
指導教授：林盈達 博士

Student：Po Han Cheng
Advisor：Dr. Ying-Dar Lin

國立陽明交通大學
網路工程研究所
碩士論文
A Thesis
Submitted to Degree Program of Computer Science
College of Computer Science
National Yang Ming Chiao Tung University
in Partial Fulfillment of the Requirements
for the Degree of
Master of Science
in
Computer Science

June 2026
Taiwan, Republic of China

中華民國 一一五年六月

誌謝

本篇論文的完成，凝聚了許多人的心力與協助。在研究的過程中，我獲得無數的啟發與幫助，才能一路走到今日，完成這段難能可貴的學術旅程。

首先，特別感謝我的指導教授林盈達。在研究初期選題與規劃階段，教授協助界定研究範疇，釐清核心問題，使研究得以順利開展。在遭遇研究瓶頸時，教授透過定期的討論給予指引，協助修正研究方向。同時，感謝教授提供穩定的研究資源與環境，使我能專注於實驗執行與數據分析，並在過程中學習如何系統性地解決問題。

感謝賴源正教授在論文撰寫過程中的細心指導，使我更能掌握論述的邏輯與節奏，得以清晰而有條理的方式呈現研究成果。也感謝游家牧教授，從技術面給予我重要的建議，協助我優化實驗設計，並在研究成果投稿過程中，提供許多實用且具建設性的意見，幫助我邁向更高的學術標準。

感謝鴻海研究院李維斌執行長及林國鈞研究員。兩位在研究期間提供專業的技術諮詢，並開放寶貴的技术資源與合作機會。透過多次深入的產學交流會議，兩位針對研究模型與系統驗證提供的反饋，協助我有效突破實驗瓶頸，讓研究成果趨於完善。

感謝實驗室同儕儒宏、宏祐與宇奐。在長時間的實驗與開發過程中，我們共同討論技術實作細節、排查系統錯誤，並交流學術與軟體開發經驗。這些互動不僅提供了實質的技術支援，也讓研究過程中的各項工作得以順利推展。

感謝家人和煒嫻長久以來的支持與包容。無論我在研究中多麼忙碌或低落，你們給我的鼓勵和陪伴始終是我不斷前行的最大動力與依靠。

最後，感謝自己在面對研究挑戰時的持續投入。透過獨立解決問題與不斷嘗試，我完成了這項研究任務。這段期間培養的抗壓性與技術能力，將成為未來進入專業領域的重要基礎。

鄭博涵 謹致於

中華民國 一一五年六月

HiPert: 針對大型語言模型程式碼場景間接提示注入攻擊之三層語法和語意感知防禦框架

學生：鄭博涵

指導教授：林盈達 博士

國立陽明交通大學 網路工程研究所 碩士班

摘 要

大型語言模型已經成為開發者常用的工具，可以協助補全程式、修改錯誤等等。然而，這些模型會同時閱讀使用者要求與外部取得的程式碼內容。如果攻擊者把惡意指令藏在註解、字串或變數名稱中，模型可能誤把這些內容當成真正的指令，進而產生不安全的程式碼。這種情況對程式碼助理特別危險，因為被污染的程式片段看起來可能仍然可以編譯，也不一定會被一般靜態檢查發現。

本文提出 HiPert，一套用於偵測並清理程式碼中間接提示注入的防禦框架。HiPert 的核心想法是先理解程式碼結構，再針對較容易藏入攻擊內容的位置進行檢查。系統會先使用 Tree-sitter 找出程式碼中註解、字串與識別符等高風險區域，接著使用三層偵測：第一層快速過濾明顯的危險樣式與異常結構；第二層偵測看起來像亂碼、但可能影響模型的對抗性片段；第三層測試高隱蔽，看起來自然的攻擊，若偵測出可疑片段，系統會移除或替換該片段，同時盡量保留原本的程式邏輯。

實驗結果顯示，在六種最新型的攻擊手法上，HiPert 的平均 F1 Score 達到 0.80，優於 CodeGarrison[1] 的 0.70、DePA[2] 的 0.51，以及 KillBadCode[3] 的 0.30。這些結果凸顯本文主要貢獻：結合程式碼結構分析與防禦框架的三層架構，來同時偵測對抗型與語意型的提示注入攻擊，在清理過後，讓不同受害模型的攻擊成功率從原本的 64%–75% 降到 10.5%–21.5%。

關鍵字：間接提示注入攻擊、大型語言模型、程式碼防禦、Tree-sitter

HiPert: A Three-Layer Syntax-Semantics-Aware Defense Against Indirect Prompt Injection in LLM-based Code Contexts

Student : Po Han Cheng

Advisor: Dr. Ying-Dar Lin

Degree Program of Computer Science
National Yang Ming Chiao Tung University

Abstract

Code large language models (LLMs) are widely used in software development, yet they remain vulnerable to indirect prompt injection attacks hidden in comments, strings, or identifiers. Although these malicious instructions do not alter program execution, they can manipulate models into generating unsafe code.

This thesis presents HiPert, a defense framework that detects and sanitizes indirect prompt injections by analyzing code structure instead of plain text. Utilizing Tree-sitter to locate high-risk regions, HiPert applies three sequential checks: explicit pattern filtering, adversarial fragment detection, and node-behavior perturbation analysis. Flagged nodes are neutralized or replaced to preserve the original executable logic.

Experiments on six attack types show that HiPert achieves a leading average F1 Score of 0.80, outperforming CodeGarrison [1], DePA [2], and KillBadCode [3]. These results highlight the main contribution: combining structural analysis with a three-layer defense framework to simultaneously detect adversarial and semantic triggers, reducing the attack success rate across different victim models from 64%–75% to 10.5%–21.5% after sanitization.

Keywords: Indirect Prompt Injection, Code LLM, Tree-sitter, Node Perturbation Analysis

Table of Contents

Chinese Abstract	i
English Abstract	ii
Table of Contents	iii
List of Figures	vi
List of Tables	vii
1 Introduction	1
2 Background and Related Work	4
2.1 Background	4
2.1.1 Indirect Prompt Injection	4
2.1.2 CST and AST	6
2.1.3 Min-K%	7
2.2 Related Work	8
2.2.1 Prompt Injection Attacks	9
2.2.2 Prompt Injection Defenses	10
2.2.3 Position of HiPert	12
3 Indirect Prompt Injection in Code Contexts	14
3.1 Notation Table	14
3.2 Problem Overview	15
3.3 Problem Statement	16

4	HiPert Framework.....	19
4.1	Solution Overview	19
4.2	Three-Layer Defense Framework	21
4.2.1	Layer 1: Syntax-Guided Pre-Filtering	22
4.2.2	Layer 2: CST-Guided Dynamic Min-K% Scoring	23
4.2.3	Layer 3: CST-Guided Node Perturbation Analysis	25
4.3	Sanitization and Cleaning	26
4.4	Illustrative Example	28
5	Implementation	30
5.1	Experimental Setup.....	30
5.1.1	Datasets.....	30
5.1.2	Models.....	31
5.1.3	Execution Environment.....	31
5.2	Implementation of Defense System.....	32
5.3	Evaluation Metrics	34
5.3.1	Precision, Recall, F1 Score, and FPR	34
5.3.2	Detection Latency	35
5.3.3	AUROC.....	36
5.4	Baselines	36
6	Evaluation	38
6.1	Parameter Setting	38
6.2	HiPert vs. SOTA Defenses.....	39
6.2.1	Main Detection Result	39
6.2.2	Threshold-Agnostic Detection Performance.....	41

6.3	One Layer vs. Three Layers.....	42
6.3.1	Independent Contribution of Each Layer.....	42
6.3.2	Cumulative Detection Performance	43
6.4	Surrogate Models vs. Black-Box Victim Models	44
6.4.1	Cross-Surrogate Generalization	44
6.4.2	Attack Mitigation on Open-Weight Victim Models.....	45
6.4.3	Evaluation of commercial black-box API models	46
6.4.4	Impact on Clean-Code Utility	46
6.5	Known Attacks vs. Adaptive and Unseen Attacks	47
6.5.1	Adaptive Attack	47
6.5.2	Transfer Attack	48
6.5.3	Leave-One-Attack-Out Evaluation	50
7	Conclusion	52
	References.....	54

List of Figures

Figure 1	Attack scenario of indirect prompt injection.	5
Figure 2	Problem Overview	16
Figure 3	Overview of the HiPert framework.	20
Figure 4	Layer 2 CST-Guided Dynamic Min-K% Scoring.	23
Figure 5	Layer 3 CST-guided node perturbation analysis.	25
Figure 6	Illustrative example of HiPert.	28
Figure 7	Execution time comparison per code sample.	40
Figure 8	ROC curve.	41
Figure 9	Cumulative layer performance.	42
Figure 10	Each layer performance.	43
Figure 11	Cross model generalization.	44
Figure 12	ASR on different victim models.	45
Figure 13	Transferability effect on ASR.	48
Figure 14	Cross-tokenizer robustness.	49

List of Tables

Table 1	Related Work on Prompt Injection Attacks	9
Table 2	Related Work on Prompt Injection Defenses	10
Table 3	Notation Table.	14
Table 4	Main Result	40
Table 5	Evaluation of commercial black-box API models	46
Table 6	Clean-code utility.	46
Table 7	Original attacks vs. adaptive attacks	48
Table 8	Defense Against DePA-Targeted Attacks	50
Table 9	Leave-one-attack-out evaluation results.	51

Chapter 1. Introduction

Code Large Language Models (Code LLMs) stream-line software engineering by assisting in code completion, debugging, and repository analysis. However, because these models process user instructions alongside untrusted external context, they introduce a critical vulnerability: attackers can embed hidden malicious instructions that compromise the model’s instruction-following behavior.

This vulnerability is known as prompt injection. While direct prompt injection involves explicit user inputs, indirect prompt injection exploits external data sources such as repositories, documentation, and pull requests [11, 17]. Once injected into the context window, Code LLMs struggle to isolate trusted instructions from untrusted text, a risk magnified in agentic workflows with local environment access [8].

In code generation, indirect prompt injection presents severe software security threats, potentially forcing models to output insecure code, create backdoors, or leak sensitive data. Consequently, effective defense mechanisms must transcend binary classification; they must precisely locate and sanitize malicious payloads before the context reaches downstream models.

Mitigating these threats remains challenging because payloads can reside in syntactically valid, non-functional regions like comments, strings, or identifiers. Attacks range from statistically anomalous optimization patterns (e.g., ShadowCode [10], INSEC [7]) to stealthy, natural-looking semantic manipulation (e.g., XOXO [6], ITGen [5], Flashboom [4]).

Standard defenses fail against this spectrum: keyword filters miss implicit payloads, while text-statistic metrics are easily bypassed by natural-looking text. This thesis introduces a frame-

work designed to resolve these limitations by focusing on high-risk syntactic regions and evaluating their explicit influence on model behavior.

We propose HiPert, a hierarchical defense framework named after its two core conceptual pillars: **H**ierarchy and **P**erturbation. HiPert utilizes concrete syntax trees to isolate high-risk structural nodes and employs a three-layer pipeline to filter explicit patterns, detect statistical anomalies, and measure semantic perturbation. Flagged nodes are automatically neutralized to safeguard downstream models without altering executable program logic.

Empirical evaluations across six attack families demonstrate HiPert’s efficacy. The framework achieves an average F1 Score of 0.80, outperforming CodeGarrison (0.70), DePA (0.51), and KillBadCode (0.30). Furthermore, sanitization reduces the attack success rate (ASR) from 64%–75% down to 10.5%–21.5% on open-weight models, and by up to 19.39 percentage points on commercial black-box coding agents.

This thesis makes three main contributions:

- **Layered Architecture:** HiPert isolates model-facing code regions and executes a three-layer detection pipeline to sanitize both structural anomalies and stealthy semantic payloads.
- **Empirical Performance:** evaluations across six attack types yield a leading average F1 Score of 0.80, outperforming state-of-the-art baselines in five of the six evaluated families.
- **Practical Deployability:** the framework functions as an inference-time pre-API sanitizer, utilizing local open-source surrogate models to clean context without requiring internal access to downstream black-box models.

Thesis Organization

The rest of this thesis is organized as follows. Chapter 2 provides the background concepts and reviews related work on prompt injection attacks and defenses. Chapter 3 introduces the scenario and problem statement of indirect prompt injection in code contexts. Chapter 4 presents the architecture and design of the proposed HiPert framework. Chapter 5 details the concrete implementation and experimental configuration. Chapter 6 reports the evaluation metrics, experimental results, and robustness analyses. Chapter 7 concludes this thesis and discusses future directions.



Chapter 2. Background and Related Work

This chapter introduces the background concepts used throughout this thesis and reviews prior work on prompt injection attacks and defenses in code-generation systems. To avoid overly deep section nesting, the chapter is organized into two groups: background concepts and related work.

2.1 Background

2.1.1 Indirect Prompt Injection

Prompt injection is an input-manipulation attack in which an LLM is induced to follow an unintended instruction. Unlike direct prompt injection, where the attacker directly writes the malicious instruction in the user prompt, indirect prompt injection places the malicious instruction in external content that the user or the system later provides to the model. In a code-generation workflow, such external content can be a GitHub repository, Stack Overflow answer, documentation page, package example, commit message, issue discussion, or RAG-retrieved source-code fragment.

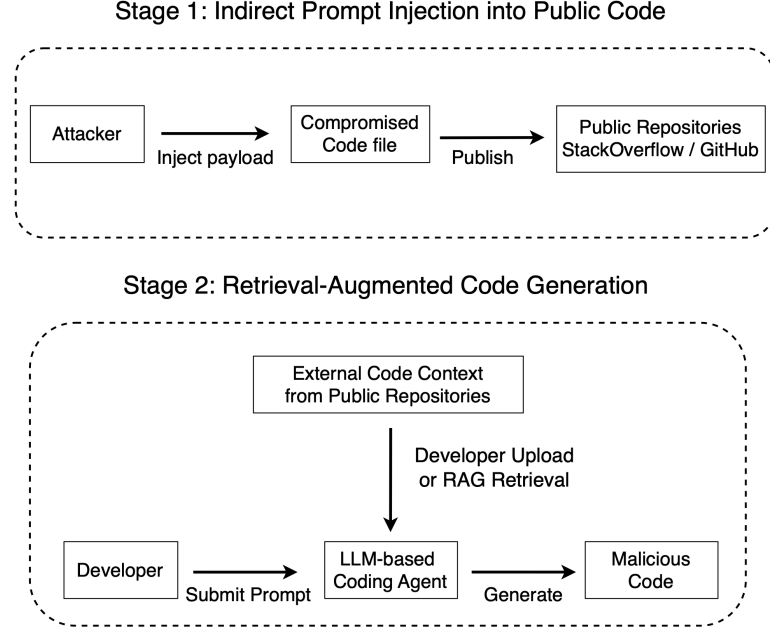


Figure 1: Attack scenario of indirect prompt injection.

Figure 1 shows the attack scenario considered in this thesis. The first stage describes the publication of the payload. An attacker injects a hidden payload into a code file and publishes the compromised file to a public or shared programming source, such as GitHub or Stack Overflow. The key property of this stage is that the attacker does not need to interact with the victim developer or the target Code LLM directly. Instead, the attacker only needs to poison a source that may later be retrieved, copied, indexed, or reused by a code-generation system.

The second stage describes retrieval-augmented code generation. A benign developer submits a normal programming request to an LLM-based coding agent. During the generation process, the system may retrieve external code context from public repositories, or the developer may manually upload or paste a code fragment as auxiliary context. The coding agent then observes the developer prompt and the external code context in the same model input. If the external context contains a hidden payload, the model may interpret that payload as an instruction rather than as untrusted data, causing the generated output to include attacker-desired malicious behavior.

The code-generation setting is especially vulnerable because source code contains regions

that are visible to the model but may have limited or indirect effect on runtime execution. Comments, string literals, identifiers, metadata, and dead-code-like snippets can all carry natural-language or semi-natural-language instructions. These regions may pass compilation and simple static checks, yet still influence the model because the model processes them as text. Therefore, indirect prompt injection in code scenarios is not only a prompt-level problem; it is also a code-context sanitization problem.

This attack scenario motivates the defense objective of this thesis. Before untrusted external code reaches the downstream Code LLM, the defense should localize high-risk code regions, detect whether they contain malicious triggers, and remove or neutralize those triggers while preserving the useful executable logic of the original code. This is why the later method focuses on syntax-guided extraction, statistical anomaly detection, and perturbation-based influence measurement.

2.1.2 CST and AST

A syntax-aware defense needs a structured representation of code rather than treating the entire file as plain text. Two common representations are the Abstract Syntax Tree (AST) and the Concrete Syntax Tree (CST). An AST represents the abstract program structure that is most relevant to compilation and semantic analysis. It usually removes or compresses lexical details such as punctuation, delimiters, comments, and some formatting tokens. This abstraction is useful for program analysis, but it is not always sufficient for prompt-injection defense because many injection payloads are placed exactly in lexical or non-executable regions.

A CST preserves the concrete syntactic form of the source code. It keeps fine-grained node spans, punctuation, delimiters, comments, string literals, and identifiers closer to their original textual positions. For example, in `int count = 0; // init`, a CST can preserve the

identifier `count`, the literal `0`, the semicolon, and the comment `// init` with precise source ranges. This precision is important because the defense must modify suspicious regions without accidentally changing unrelated executable logic.

In the scope of this thesis, a CST node refers to a concrete, structurally isolated syntactic unit generated by the parser, which maps directly to a continuous byte range in the source code. Unlike flat text tokens, a CST node captures both the lexical content and its syntactic context (e.g., distinguishing a string literal from a comment block), which serves as the fundamental entity for our granularity analysis.

In this thesis, Tree-sitter is used as the parser for CST construction. Tree-sitter is suitable for this setting for three reasons. First, it supports many programming languages, making the defense applicable to heterogeneous code contexts. Second, it provides byte-level and node-level spans, which allow model-facing regions to be aligned with token-level losses or logits-based scores. Third, its incremental parsing and error recovery are useful when external code is incomplete, malformed, or intentionally obfuscated.

Although AST and CST are both tree-based code representations, this thesis mainly relies on CST-guided analysis because the target regions include comments, strings, identifiers, and decoy-like fragments. These regions are precisely the places where an attacker can hide prompt-injection payloads while keeping the code syntactically plausible. CST-guided localization therefore provides the structural basis for the later dynamic Min-K% scoring and node perturbation analysis.

2.1.3 Min-K%

Language models assign a probability to each token conditioned on its previous context. A token that the model considers unlikely has a high negative log-likelihood. Traditional

perplexity-based detection averages this signal over a whole sequence. This average is useful for detecting globally unnatural text, but it can miss short malicious payloads because a few abnormal tokens may be diluted by a large amount of benign code.

Min-K% addresses this dilution problem by focusing on the least likely portion of the token sequence. In practice, the method ranks token-level losses and computes the score from the tokens with the largest negative log-likelihood values. This makes the detector more sensitive to localized abnormal fragments, such as adversarial suffixes, random-looking strings, or optimized perturbations hidden inside comments and strings.

In code-generation attacks, Min-K% is especially relevant to payloads produced by black-box or optimization-based methods such as ShadowCode and INSEC. These attacks often generate non-natural perturbation strings that may not be semantically meaningful to humans but can strongly influence the Code LLM. Such strings tend to create local probability anomalies, so a node-level Min-K% score can expose them more effectively than file-level perplexity.

However, Min-K% is not sufficient for all indirect prompt injection attacks. Semantic triggers such as XOXO and ITGen can use natural-looking identifiers or semantics-preserving transformations. These triggers may have low perplexity and may not appear statistically abnormal. Therefore, this thesis uses Min-K% as the second layer for adversarial or high-surprise payloads, and complements it with CST-guided node perturbation analysis for low-perplexity but high-influence semantic triggers.

2.2 Related Work

Table 1 summarizes representative prompt injection attacks, while Table 2 compares existing defense methods with our proposed approach.

Table 1: Related Work on Prompt Injection Attacks

Attack	Category	External-context IPI	Code-local carrier	Low-runtime impact carrier	Semantic/stealthy trigger	Workflow-level target
MalwareBench [19]	Direct jailbreak	–	–	–	–	–
MOCHA [9]	Multi-turn prompts	–	–	–	△	✓
PR-Attack [17]	RAG poisoning	✓	△	–	△	✓
CodeJailbreaker [16]	Metadata injection	✓	△	✓	✓	△
AIShellJack [8]	Agentic editor	✓	△	△	–	✓
ShadowCode [10]	Non-functional payloads	✓	✓	✓	△	–
INSEC [7]		✓	✓	✓	△	–
XOXO [6]	Semantic transform	✓	✓	✓	✓	✓
ITGen [5]		✓	✓	✓	✓	–
Flashboom [4]	Auditor deception	✓	✓	✓	✓	✓
CoTDceptor [20]		✓	✓	✓	✓	✓

2.2.1 Prompt Injection Attacks

Early research mainly focused on direct jailbreak attacks, where the attacker explicitly designs prompts to bypass safety policies and induce malicious code generation [19]. Later attacks became more indirect because LLM systems increasingly consume external context. PR-Attack poisons the RAG database and coordinates the poisoned document with a prompt-side trigger [17]. CodeJailbreaker hides malicious intent in commit messages, while AIShellJack targets high-privilege agentic coding editors by placing malicious instructions in external development resources [16, 8]. These attacks demonstrate practical injection surfaces, but their triggers still depend on recognizable instruction-like content or system-level agent behaviors. In contrast, our work focuses on localizing and sanitizing malicious regions inside source-code context before the code is used by the victim model.

Recent code-oriented attacks exploit syntax-sensitive locations inside code. ShadowCode and INSEC inject optimized non-functional perturbations or adversarial comment strings into code context [10, 7]. These attacks are effective in black-box settings, but their perturbations often create localized statistical anomalies. Therefore, a syntax-aware detector can narrow the search space to comments, strings, and other non-executed regions and apply dynamic Min-K% analysis more precisely than file-level perplexity. XOXO and ITGen are harder because they use

Table 2: Related Work on Prompt Injection Defenses

Paper	Category	Attack type	Code Gen	Black box victim	Solution						
					Static code analysis	Node perturb. analysis	PPL	Min-K%	Redesign agent workflow	RL	Fine-tuning
UniGuardian [15]	General IPI	Direct									V
Instruct Detector [12]		Indirect									V
Self-RedTeam [14]		Direct								V	
BIPIA [11]		Indirect									V
Chen et al. [13]		Indirect									V
SecAlign [21]		Indirect									V
StruQ [22]		Indirect									V
CaMeL [23]	Agent IPI	Indirect							V		
DePA [2]	Code poisoning	X	V	V			V				
KillBadCode [3]		X	V	V			V				
CodeGarrison [1]		X	V	V							V
Ours	Code context IPI	Indirect	V	V	V	V		V			

semantics-preserving code transformations or identifier replacement to create natural-looking triggers [6, 5]. Their limitation, from a defender’s perspective, is that the trigger must still strongly influence the model’s generation behavior. HiPert uses CST-guided node perturbation to expose this influence even when the trigger has low perplexity.

Flashboom and CoTDeceptor further show that LLM-based code auditors can be misled by attention diversion, decoy snippets, or CoT-aware obfuscation [4, 20]. These works highlight weaknesses of LLM reasoning and auditing, but they do not provide a general inference-time sanitization mechanism for untrusted code context. Compared with them, HiPert is defensive rather than generative: it combines CST localization, anomaly scoring, and perturbation-based influence measurement to remove high-risk nodes while preserving benign code functionality.

2.2.2 Prompt Injection Defenses

Existing defenses use several types of signals. Generic prompt-injection defenses attempt to distinguish instructions from data through benchmark-driven mitigation, instruction detection, prompt segmentation, or trained detectors [11, 12, 13, 15]. Reinforcement-learning approaches such as Self-RedTeam improve model robustness through adversarial self-play [14]. However,

these methods are mostly designed for natural-language prompts or general LLM safety. They do not explicitly model code syntax, so they may miss payloads hidden in identifiers, comments, strings, or dead code.

Recent training-based defenses more directly address indirect prompt injection. SecAlign formulates prompt-injection defense as a preference-optimization problem: it constructs preference pairs containing secure and insecure responses for prompt-injected inputs, and then optimizes the model to prefer the secure response [21]. StruQ uses structured queries to separate trusted instructions from untrusted data, combining a secure front-end with a structured instruction-tuned model that learns to follow only the instruction channel [22]. These methods improve model-level robustness, but they require defensive training, specialized formatting, or changes to the defended model behavior. They also do not localize malicious regions inside source code, so they are less suitable when the defense must sanitize untrusted code context before calling a black-box victim model.

System-level defenses redesign the agent execution environment instead of only changing the prompt or model. CaMeL creates a protective layer around an LLM agent, extracts trusted control and data flows, and uses capability-based checks to prevent untrusted data from changing program flow or causing unauthorized data exfiltration [23]. This direction is valuable for tool-using agents, but it assumes the deployment can redesign the agent runtime and tool-invocation policy. It is not designed as a syntax-aware detector for comments, strings, identifiers, or decoy snippets in code-generation context.

Code-specific defenses are closer to this thesis but still have important limitations. DePA performs line-level perplexity analysis for dead-code poisoning and compares each line with the file-level perplexity distribution [2]. This is more code-aware than pure word-level perplexity, but the line remains a coarse unit: a malicious identifier, comment fragment, or string literal

can be diluted by surrounding benign code. KillBadCode uses code naturalness to identify trigger tokens whose deletion makes poisoned code more natural [3]. This assumption works for unnatural triggers, but it is weaker against XOXO and ITGen-style semantic triggers because a renamed identifier can be both natural and maliciously influential. CodeGarrison learns hybrid code representations to detect poisoned source-code samples [1]. Its learned features are richer than simple perplexity, but its performance depends on available training data and known attack distributions, so it can be brittle against unseen prompt-injection styles.

2.2.3 Position of HiPert

HiPert addresses these limitations by treating source code as structured input rather than flat text. First, Tree-sitter narrows the search space to high-risk CST nodes such as comments, strings, identifiers, and decoy-like snippets. This design directly follows the attack scenario in Figure 1, where the malicious payload is embedded in external code context before the context is consumed by the coding agent. Second, CST-guided dynamic Min-K% targets high-perplexity adversarial payloads such as those used by ShadowCode and INSEC. Third, CST-guided node perturbation measures whether removing or neutralizing a node causes a large logits-level behavior change, which is necessary for low-perplexity semantic attacks such as XOXO and IT-Gen.

Compared with SecAlign and StruQ, HiPert does not require aligning, fine-tuning, or replacing the victim model; it can operate as an inference-time pre-API sanitizer for both white-box and black-box victim models. Compared with CaMeL, HiPert does not require redesigning the agent runtime, tool policy, or control/data-flow architecture; it focuses on sanitizing untrusted code context before the context reaches the downstream Code LLM. Compared with DePA, HiPert is finer-grained than line-level detection because it operates on syntax nodes rather than

complete lines. Compared with KillBadCode, it does not rely only on code-naturalness improvement, so it is less dependent on the assumption that the malicious trigger must look unnatural. Compared with CodeGarrison, it avoids depending entirely on a supervised detector trained on known attack distributions. This positioning makes HiPert better suited to inference-time defense against heterogeneous indirect prompt injection attacks in code-generation scenarios.



Chapter 3. Indirect Prompt Injection in Code Contexts

3.1 Notation Table

Table 3 summarizes the notation used in this chapter. The table is organized by category so that each symbol is tied to its role in the problem formulation.

Table 3: Notation Table.

Category	Name	Notation	Note
Input	User Prompt	P^u	$P^u = \{u_1, u_2, \dots, u_q\}$
	External Code Files	C	$C = \{\text{row}_1, \text{row}_2, \dots, \text{row}_m\}$
	Complete Model Input	x	$x = (P^u, C)$
Code Region	Executable Code Region	C_{exec}	$C = C_{\text{exec}} \cup C_{\text{risk}}, C_{\text{exec}} \cap C_{\text{risk}} = \emptyset$
	High-risk Code Region	C_{risk}	$C_{\text{risk}} = C_{\text{comment}} \cup C_{\text{string}} \cup C_{\text{identifier}} \cup C_{\text{decoy}}$
	Comment Region	C_{comment}	$C_{\text{comment}} \subseteq C_{\text{risk}}$
	String-literal Region	C_{string}	$C_{\text{string}} \subseteq C_{\text{risk}}$
	Identifier Region	$C_{\text{identifier}}$	$C_{\text{identifier}} \subseteq C_{\text{risk}}$
Model	Code LLM / Surrogate Model	f_θ	$f_\theta : x \mapsto \tilde{y}$
	Generated Output	$\tilde{y}$	$\tilde{y} = f_\theta(P^u, \tilde{C})$
	Safe Output	y_{safe}	$f_\theta(P^u, C \setminus S^*) = y_{\text{safe}}$
	Malicious Output	y_{mal}	$f_\theta(P^u, C) = y_{\text{mal}}$
Attack	Candidate Suspicious Nodes	S	$S \subseteq C_{\text{risk}}$
	Ideal Malicious Trigger Set	S^*	$S^* \subseteq C_{\text{risk}}$
Defense	Detected Suspicious Nodes	$\hat{S}$	$\hat{S} \subseteq C_{\text{risk}}$
	Sanitized Code Context	$\tilde{C}$	$\tilde{C} = \text{Sanitize}(C, \hat{S})$
	Defense Output	o	$o = (a, \hat{S}, \tilde{C})$
	Detection Decision	a	$a \in \{\text{benign}, \text{malicious}\}$
Scoring	Logits Distribution	$\ell(x)$	$\ell(P^u, C)$ denotes the model logits before sanitization.
	Logits Distance Function	$d(\cdot, \cdot)$	$d(\ell(P^u, C), \ell(P^u, C \setminus S))$
	Detection Threshold	τ	$d(\ell(P^u, C), \ell(P^u, C \setminus S)) > \tau$
	Perturbation Budget	m	$ S \leq m$
Objective	Optimal Detected Trigger Set	$\hat{S}$	$\hat{S} = \arg \max_{S \subseteq C_{\text{risk}}, S \leq m} d(\ell(P^u, C), \ell(P^u, C \setminus S))$

The symbol P^u denotes the trusted developer instruction. In contrast, C denotes the external

code files, which may come from repositories, documentation, issue reports, or other untrusted sources. The complete model input is therefore written as $x = (P^u, C)$.

The code context C is divided into C_{exec} and C_{risk} . The set C_{exec} contains executable program logic, including control flow, statements, and computations. The set C_{risk} contains model-facing regions that are easier for an attacker to manipulate without directly changing the program’s runtime behavior. In this thesis, C_{risk} mainly consists of comments, string literals, identifiers, and decoy code regions, denoted by C_{comment} , C_{string} , $C_{\text{identifier}}$, and C_{decoy} , respectively.

The function f_θ represents the Code LLM used for generation or the surrogate model used for defense-side scoring. Given an input x , the model generates an output y . When the context is clean, the expected output is denoted as y_{safe} . When a hidden trigger successfully manipulates the model, the output becomes the attacker-desired malicious output y_{mal} .

The set S represents candidate suspicious nodes selected from C_{risk} . The ideal malicious trigger subset is denoted as S^* , while $\hat{S}$ is the subset detected by the defense system. After sanitization, the original code context C becomes $\tilde{C}$. To measure the influence of a candidate node set, this thesis uses the model logits $\ell(x)$ and compares two logits distributions with a distance function $d(\cdot, \cdot)$. The threshold τ is used to decide whether a node is suspicious, and m limits the perturbation search space.

3.2 Problem Overview

Figure 2 summarizes the overall problem flow. Given an untrusted input $x = (P^u, C)$, the defense first detects whether the code context is benign or malicious and predicts suspicious nodes $\hat{S}$. If the input is benign, the model can directly produce a safe output. If the input is malicious, the suspicious context is sanitized into $\tilde{C}$ before being sent to the LLM, so the final

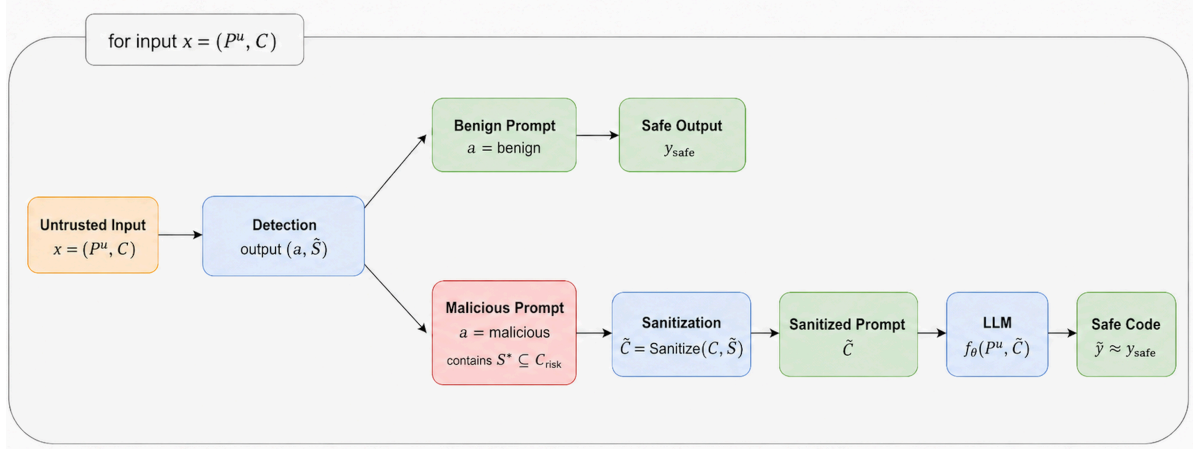


Figure 2: Problem Overview

generated code $\tilde{y}$ remains close to the safe output y_{safe} .

3.3 Problem Statement

Input. The input consists of a developer prompt P^u and an external code context C :

$$x = (P^u, C). \quad (3.1)$$

The code context is partitioned into executable and high-risk regions:

$$C = C_{\text{exec}} \cup C_{\text{risk}}, \quad C_{\text{exec}} \cap C_{\text{risk}} = \emptyset. \quad (3.2)$$

The executable region C_{exec} contains the core program logic. The high-risk region C_{risk} contains code elements that are visible to the model and may carry hidden instructions:

$$C_{\text{risk}} = C_{\text{comment}} \cup C_{\text{string}} \cup C_{\text{identifier}} \cup C_{\text{decoy}}. \quad (3.3)$$

Output. The defense system outputs three objects:

$$o = (a, \hat{S}, \tilde{C}), \quad (3.4)$$

where $a \in \{\text{benign}, \text{malicious}\}$ is the detection decision, $\hat{S} \subseteq C_{\text{risk}}$ is the detected suspicious node set, and $\tilde{C}$ is the sanitized code context. The sanitized context is produced by removing or neutralizing the detected nodes:

$$\tilde{C} = \text{Sanitize}(C, \hat{S}). \quad (3.5)$$

The downstream model then generates code from the sanitized input:

$$\tilde{y} = f_{\theta}(P^u, \tilde{C}). \quad (3.6)$$

Objective. The attacker attempts to create a hidden trigger subset $S^* \subseteq C_{\text{risk}}$ such that the polluted context changes the model output from a safe output to a malicious output:

$$f_{\theta}(P^u, C) = y_{\text{mal}}, \quad f_{\theta}(P^u, C \setminus S^*) = y_{\text{safe}}. \quad (3.7)$$

The defender aims to approximate this hidden trigger subset by finding influential suspicious nodes in C_{risk} :

$$\hat{S} = \arg \max_{S \subseteq C_{\text{risk}}, |S| \leq m} d(\ell(P^u, C), \ell(P^u, C \setminus S)). \quad (3.8)$$

This objective captures the main design principle of this thesis: a malicious trigger should be both located in a high-risk code region and influential to the model's prediction. After detecting $\hat{S}$, the defense should reduce the attack success rate of the downstream model while preserving the utility of benign code.

Constraints. The problem is subject to the following constraints.

1. **High-risk localization constraint.** The defense should search primarily within C_{risk} instead of modifying arbitrary tokens in the whole file:

$$\hat{S} \subseteq C_{\text{risk}}. \quad (3.9)$$

2. **Executable-logic preservation constraint.** Sanitization should not alter the core executable behavior of the original code. The executable region should remain unchanged unless a later security policy explicitly requires otherwise:

$$C_{\text{exec}}(C) = C_{\text{exec}}(\tilde{C}). \quad (3.10)$$

3. **Minimality constraint.** The detected suspicious set should be small enough to avoid unnecessary modification of benign context:

$$|\hat{S}| \leq m. \quad (3.11)$$

4. **Deployability constraint.** The defense should remain applicable when the victim model is black-box. If victim logits are unavailable, the logits $\ell(\cdot)$ used for scoring can be estimated with a local open-source surrogate model before $\tilde{C}$ is sent to the victim model.
5. **Latency constraint.** Exhaustively perturbing all possible node subsets is infeasible. The defense must approximate the objective through syntax-guided extraction, lightweight filtering, adversarial-fragment scoring, and selective node perturbation.

Chapter 4. HiPert Framework

4.1 Solution Overview

Chapter 3 formalizes the defense objective as detecting prompt-injection triggers embedded in untrusted code context and producing a cleaned context before the downstream Code LLM consumes it. This chapter presents **HiPert**, a three-layer syntax–semantics-aware defense framework for indirect prompt injection in LLM-based code scenarios. HiPert is designed around one key observation: in code-context prompt injection, the attacker usually hides the payload in regions that are highly visible to the LLM but are less likely to be checked carefully by developers, such as comments, string literals, and variable or identifier names. These regions may not directly alter the executable control flow, yet they can still steer the LLM’s instruction-following behavior.

HiPert therefore avoids treating the entire code file as plain text. Instead, it first parses the external code into a concrete syntax tree (CST), extracts high-risk non-executable or model-facing nodes, and then applies three increasingly expensive guardrails. Layer 1 quickly filters explicit or structurally suspicious payloads. Layer 2 detects adversarial random-looking payloads through CST-guided dynamic Min-K% token-loss scoring. Layer 3 handles natural-looking semantic triggers by neutralizing candidate nodes and measuring their influence on the model’s output distribution. The pipeline uses an early-exit design: once a node is detected by an earlier layer, it is sanitized immediately and does not need to pass through the later, more expensive analysis.

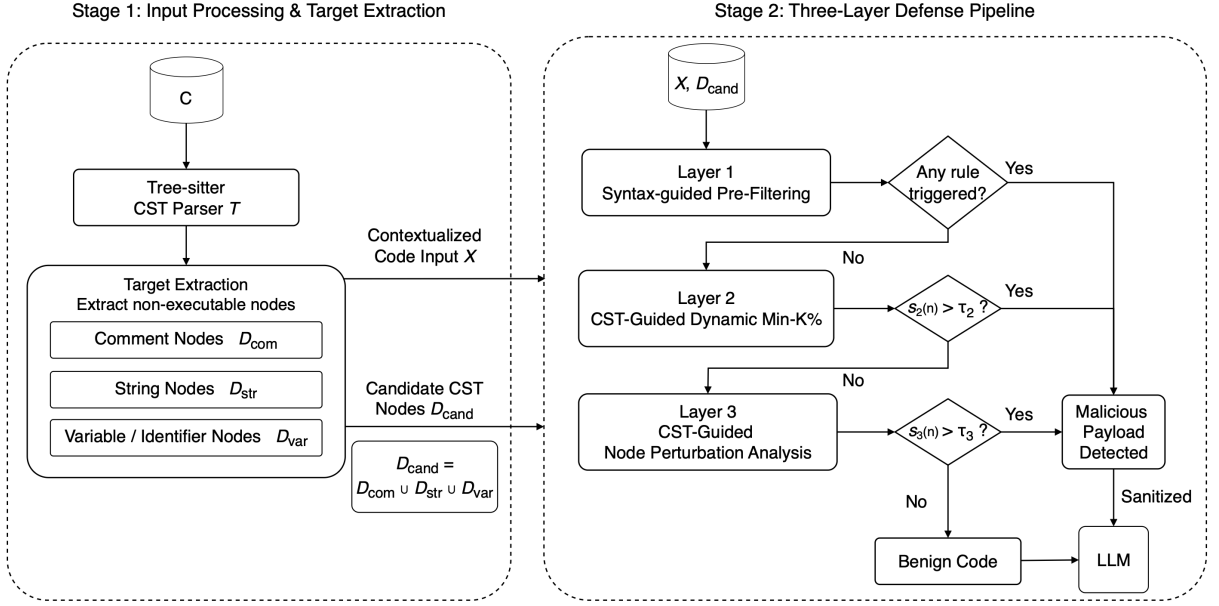


Figure 3: Overview of the HiPert framework.

Figure 3 summarizes the updated workflow. The first stage is **input processing and target extraction**. Given an external code context C , HiPert invokes a Tree-sitter CST parser T :

$$X = T(C), \quad (4.1)$$

where X denotes the contextualized code input enriched with CST node type, node span, and source position information. The target extraction module then identifies candidate nodes from three risk regions:

$$\mathcal{D}_{\text{cand}} = \mathcal{D}_{\text{com}} \cup \mathcal{D}_{\text{str}} \cup \mathcal{D}_{\text{var}}, \quad (4.2)$$

where $\mathcal{D}_{\text{com}}$ contains comment nodes, $\mathcal{D}_{\text{str}}$ contains string-literal nodes, and $\mathcal{D}_{\text{var}}$ contains variable or identifier nodes. Each candidate node $n \in \mathcal{D}_{\text{cand}}$ is represented as

$$n = (t_n, \text{span}(n), \text{text}(n)), \quad (4.3)$$

where t_n is the CST node type, $\text{span}(n)$ is the byte or token span in C , and $\text{text}(n)$ is the original node text.

The second stage is the **three-layer defense pipeline**. Let $\mathcal{D}^{(0)} = \mathcal{D}_{\text{cand}}$ be the initial

candidate set. Layer 1 produces a detected set $\hat{\mathcal{S}}_1$ and a passed set $\mathcal{D}^{(1)}$:

$$\hat{\mathcal{S}}_1 = \{n \in \mathcal{D}^{(0)} : S_1(n) > \tau_1\}, \quad \mathcal{D}^{(1)} = \mathcal{D}^{(0)} \setminus \hat{\mathcal{S}}_1. \quad (4.4)$$

Layer 2 only evaluates nodes that pass Layer 1:

$$\hat{\mathcal{S}}_2 = \{n \in \mathcal{D}^{(1)} : S_2(n) > \tau_2\}, \quad \mathcal{D}^{(2)} = \mathcal{D}^{(1)} \setminus \hat{\mathcal{S}}_2. \quad (4.5)$$

Layer 3 then evaluates the remaining hard cases:

$$\hat{\mathcal{S}}_3 = \{n \in \mathcal{D}^{(2)} : S_3(n) > \tau_3\}. \quad (4.6)$$

The final malicious-node set is

$$\hat{\mathcal{S}} = \hat{\mathcal{S}}_1 \cup \hat{\mathcal{S}}_2 \cup \hat{\mathcal{S}}_3. \quad (4.7)$$

If $\hat{\mathcal{S}} = \emptyset$, the input is treated as benign and forwarded unchanged. If $\hat{\mathcal{S}} \neq \emptyset$, only the detected node spans are sanitized, and the cleaned code context is forwarded to the downstream Code LLM. Since HiPert operates before the victim model API call, the protected victim can be either an open-weight model or a commercial black-box model. When victim logits are unavailable, HiPert computes Layer 2 and Layer 3 scores with a local surrogate model and forwards only the sanitized code context to the victim.

4.2 Three-Layer Defense Framework

The three layers are complementary rather than redundant. Layer 1 captures obvious prompt-injection patterns and disconnected decoy code with negligible overhead. Layer 2 captures statistically abnormal adversarial suffixes that may not contain explicit malicious keywords. Layer 3 captures natural-looking semantic triggers that have low perplexity but high influence over the model’s final logits.

4.2.1 Layer 1: Syntax-Guided Pre-Filtering

Layer 1 is a fast pre-filter for obvious or structurally suspicious payloads. It does not attempt to deeply interpret the semantic effect of a node. Instead, it uses CST node boundaries to run three rule checkers on each candidate node: a regex matcher, a lexical-structural checker, and an interaction graph analyzer.

Regex matcher. The regex matcher searches for high-confidence prompt-injection cues, including instruction-overriding phrases, hidden role-control directives, suspicious command templates, and payload fragments that explicitly ask the model to ignore or bypass prior instructions. The matcher is deliberately conservative: it is intended to catch clear attacks with low latency, not to replace the later semantic layers.

Lexical-structural checker. The lexical-structural checker captures node-level irregularities such as invisible Unicode characters, excessive non-ASCII symbols, abnormal punctuation density, repeated control tokens, extremely long continuous tokens, or malformed-looking fragments inside comments and strings. To reduce false positives on short benign identifiers such as `i`, `x`, or `id`, HiPert uses smoothed ratios:

$$R_{\text{lex}}(n) = \sum_{j=1}^m \alpha_j \frac{c_j(n) + \lambda}{|n| + \Lambda}, \quad (4.8)$$

where $c_j(n)$ is the count of the j -th lexical feature, $|n|$ is the node length, λ and Λ are smoothing constants, and α_j is the feature weight. This prevents one unusual character in a short benign identifier from dominating the score.

Interaction graph analyzer. The interaction graph analyzer is designed for decoy-style attacks such as Flashboom. These attacks inject code or identifiers that look salient to the LLM

but have little or no interaction with the main program logic. HiPert constructs a lightweight interaction graph

$$\mathcal{G}_X = (V_X, E_X), \quad (4.9)$$

where vertices are CST regions or identifiers, and edges represent function-call relations, shared identifier usage, data-flow coupling, or control-flow adjacency. A node is suspicious when it is salient but isolated from the main logic:

$$R_{\text{graph}}(n; X) = \mathbf{1} \left[\text{salience}(n) \geq \tau_{\text{sal}} \wedge \text{conn}_{\text{main}}(n; \mathcal{G}_X) = 0 \right]. \quad (4.10)$$

This allows HiPert to remove dead decoys before invoking model-based analysis.

4.2.2 Layer 2: CST-Guided Dynamic Min-K% Scoring

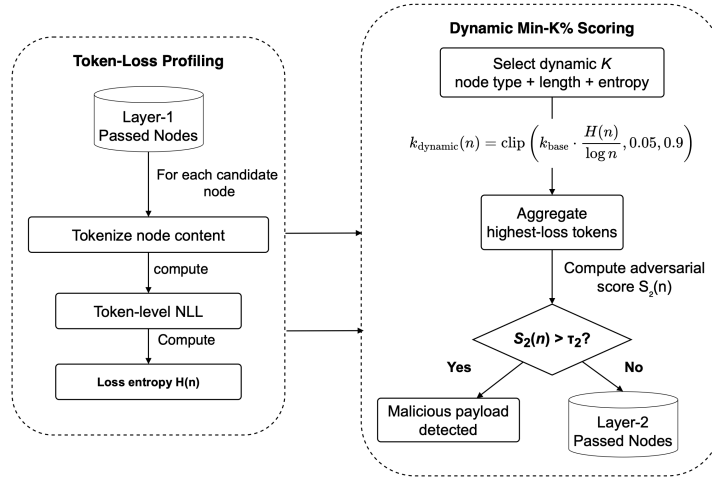


Figure 4: Layer 2 CST-Guided Dynamic Min-K% Scoring.

Layer 2 targets adversarial prompt-injection payloads that are syntactically valid but statistically unnatural, such as optimized random-looking suffixes hidden in comments or strings. Such payloads may be diluted by normal surrounding code if the detector computes file-level perplexity. HiPert therefore performs token-loss profiling at the CST-node level and aggregates only the most suspicious high-loss tokens.

As shown in Figure 4, the input of Layer 2 is the Layer 1 passed set $\mathcal{D}^{(1)}$. For a candidate

node n with token sequence $z_1, \dots, z_{|n|_{\text{tok}}}$, HiPert computes token-level negative log-likelihood:

$$\ell_i(n) = -\log p_\theta(z_i \mid X, z_{<i}), \quad i = 1, \dots, |n|_{\text{tok}}. \quad (4.11)$$

The node-loss vector is

$$\mathbf{L}(n) = \{\ell_1(n), \ell_2(n), \dots, \ell_{|n|_{\text{tok}}}(n)\}. \quad (4.12)$$

HiPert then computes the entropy of the token-loss distribution:

$$H(n) = -\sum_i p_i(n) \log p_i(n), \quad p_i(n) = \frac{\ell_i(n)}{\sum_j \ell_j(n)}. \quad (4.13)$$

The dynamic ratio is selected according to node type, length, and entropy:

$$k_{\text{dynamic}}(n) = \text{clip} \left(k_{\text{base}}(t_n) \cdot \frac{H(n)}{\log |n|_{\text{tok}}}, 0.05, 0.9 \right), \quad (4.14)$$

where $k_{\text{base}}(t_n)$ is the base ratio for the CST node type. The term $H(n)/\log |n|_{\text{tok}}$ normalizes entropy by node length, so short identifiers, medium-length strings, and long comments do not share a brittle fixed k value. In implementation, a small numerical guard is applied when a node contains very few tokens.

The first anomaly feature is the mean of the highest-loss tokens selected by k_{dynamic} :

$$\text{MeanTop}_{k_{\text{dynamic}}}(n) = \frac{1}{|\mathcal{K}(n)|} \sum_{i \in \mathcal{K}(n)} \ell_i(n), \quad (4.15)$$

where $\mathcal{K}(n)$ is the index set of the top- k_{dynamic} fraction of token losses. To prevent an attacker from hiding a short abnormal span inside long benign text, HiPert also measures the maximum sliding-window loss:

$$\text{MaxWindowLoss}(n) = \max_a \frac{1}{w} \sum_{i=a}^{a+w-1} \ell_i(n), \quad (4.16)$$

and the largest adjacent loss spike:

$$\text{MaxSpike}(n) = \max_{i>1} |\ell_i(n) - \ell_{i-1}(n)|. \quad (4.17)$$

The final Layer 2 adversarial score follows the updated scoring rule:

$$S_2(n) = \max \left(\text{MeanTop}_{k_{\text{dynamic}}}(n), \alpha \cdot \text{MaxWindowLoss}(n) + (1-\alpha) \cdot \text{MaxSpike}(n) \right). \quad (4.18)$$

The node is detected as an adversarial payload when

$$S_2(n) > \tau_2. \quad (4.19)$$

4.2.3 Layer 3: CST-Guided Node Perturbation Analysis

Layer 3 targets natural-looking semantic triggers. These payloads can be ordinary-looking variable names, comments, or strings. They may not contain suspicious keywords and may not produce high token loss, but they can still strongly influence the downstream model’s logits. The goal of Layer 3 is therefore not to ask whether a node looks strange, but whether the model’s behavior changes substantially when the node is neutralized.

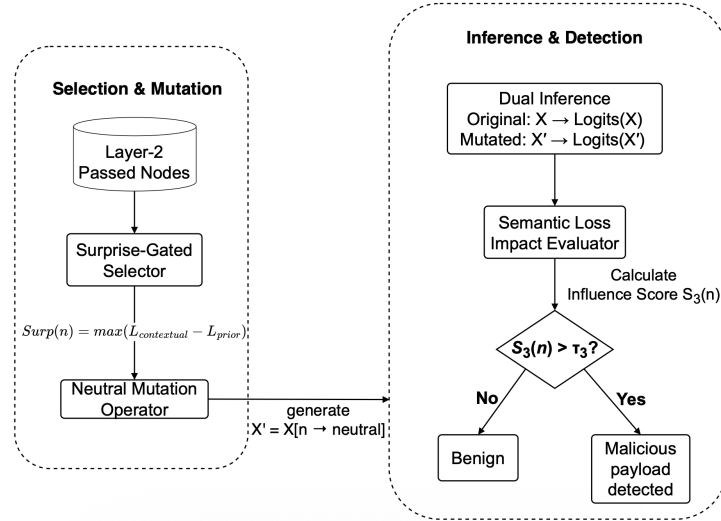


Figure 5: Layer 3 CST-guided node perturbation analysis.

Figure 5 shows the Layer 3 workflow. To reduce overhead, HiPert first uses a surprise-gated selector. For each remaining candidate node $n \in \mathcal{D}^{(2)}$, the surprise score is

$$\text{Surp}(n) = \max (L_{\text{contextual}}(n) - L_{\text{prior}}(n)), \quad (4.20)$$

where $L_{\text{contextual}}(n)$ is the node loss under the full code context, and $L_{\text{prior}}(n)$ is the prior or context-independent node loss. A high value indicates that the node becomes unusually influential or surprising when placed in its surrounding code context. Only nodes that pass this gate are sent to neutral mutation.

For a selected node n , HiPert constructs a mutated input by replacing the node with a neutral value while preserving syntax:

$$X' = X[n \rightarrow \text{neutral}]. \quad (4.21)$$

For example, a suspicious identifier is renamed to a neutral placeholder such as `tmp_var`, a suspicious string is replaced by an empty literal, and a suspicious comment span is removed. HiPert then performs dual inference:

$$X \rightarrow \text{Logits}(X), \quad X' \rightarrow \text{Logits}(X'). \quad (4.22)$$

Let $f(\cdot)$ denote the model’s final-layer logits and let $L(\cdot)$ denote the semantic loss impact computed from those logits. The updated influence score is

$$S_3(n) = L(f(X)) - \mathbb{E}_{X' \sim \text{Mut}(X,n)} [L(f(X'))]. \quad (4.23)$$

A large positive $S_3(n)$ means that the original node contributes substantially more to the risky semantic direction than its neutralized variants. Layer 3 detects the node when

$$S_3(n) > \tau_3. \quad (4.24)$$

4.3 Sanitization and Cleaning

After the three layers produce the detected node set $\hat{\mathcal{S}}$, HiPert sanitizes only the suspicious node spans. The objective is not to rewrite or repair the program. Instead, HiPert removes the LLM-facing trigger while preserving the surrounding executable-area code as much as possible.

Formally, sanitization is defined as

$$\tilde{C} = \text{Sanitize}(C, \hat{\mathcal{S}}) = C \left[\text{span}(n) \leftarrow R(n), \forall n \in \hat{\mathcal{S}} \right], \quad (4.25)$$

where $R(n)$ is a node-type-specific replacement function. Replacements are applied in descending byte-offset order:

$$\text{order}(\hat{\mathcal{S}}) = \text{sort}_{\text{desc}} \left(\{\text{start}(n) : n \in \hat{\mathcal{S}}\} \right), \quad (4.26)$$

so editing a later span does not shift the span of an earlier node.

The replacement policy follows the node type:

- **Comment nodes.** Suspicious comment spans are removed or replaced with a neutral marker. Since comments are model-facing but non-executable, this removes injected instructions without changing runtime behavior.
- **String nodes.** Suspicious string literals are replaced with syntactically valid neutral literals such as `" "`. This prevents a malicious instruction embedded in a string from being interpreted by the LLM as an active command.
- **Identifier nodes.** Suspicious identifiers are renamed to neutral placeholders such as `tmp_var`. When the identifier is used multiple times in the same scope, the renaming is applied scope-consistently to preserve syntax and references.

The final code context sent to the downstream model is

$$C_{\text{clean}} = \begin{cases} C, & \text{if } \hat{\mathcal{S}} = \emptyset, \\ \tilde{C}, & \text{otherwise.} \end{cases} \quad (4.27)$$

This makes HiPert suitable for black-box API settings. The local defense model computes

$S_2(n)$ and $S_3(n)$ before the API call, while the victim model receives only C_{clean} .

4.4 Illustrative Example

This section illustrates how HiPert prevents an adversarial identifier from misleading a downstream Code LLM during vulnerability detection. Instead of reproducing the full program, Figure 6 summarizes the key transformation: an adversarially named identifier biases the victim model toward a false negative, while HiPert neutralizes the identifier and preserves the vulnerability-bearing code structure.

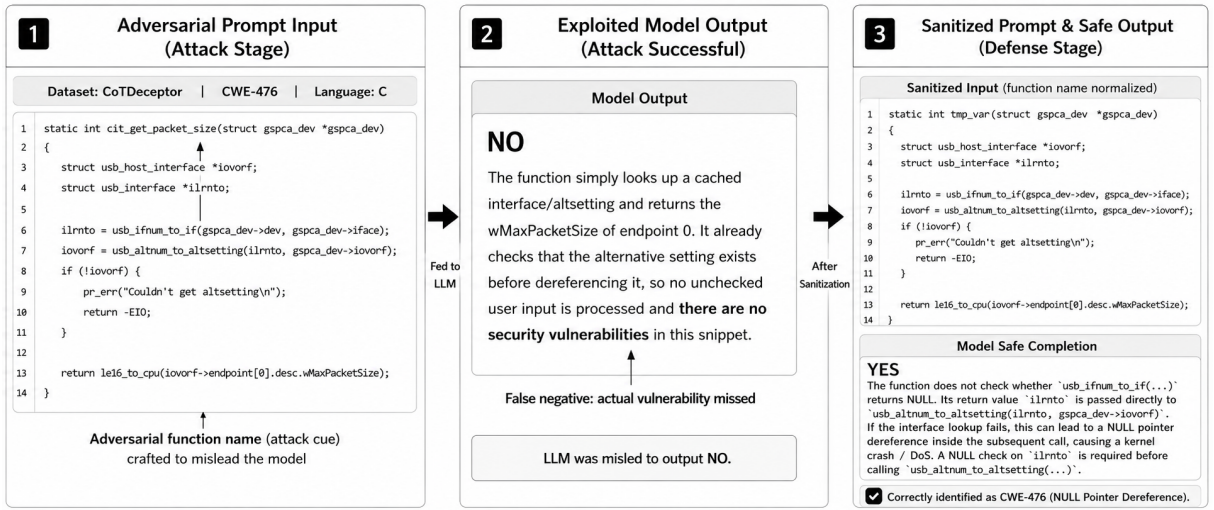


Figure 6: Illustrative example of HiPert.

Attack stage. The adversarial cue is embedded in the identifier name `cit_get_packet_size`. Although renaming this identifier does not change program semantics, it changes the lexical evidence seen by the LLM. Because code LLMs often use identifier names as high-level semantic summaries of program intent, a benign-looking name can create a misleading prior that the function is only performing a routine packet-size lookup. In the exploited prompt, this cue distracts the model from the actual pointer-safety condition and leads it to answer NO.

Missed vulnerability. The model output is a false negative. The real issue is that `usb_ifnum_to_if(...)` may return NULL, but its return value is passed directly into `usb_altnum_to_altsetting(...)` without a prior null check. If the interface lookup

fails, the subsequent call may dereference a null pointer and cause a kernel crash or denial of service. Therefore, the correct vulnerability label is YES, corresponding to CWE-476.

Defense stage. HiPert uses CST-guided perturbation analysis to identify the suspicious identifier whose neutralization strongly changes the model’s output direction. The sanitization module then applies scope-preserving alpha-renaming and replaces the adversarial name with a neutral placeholder such as `tmp_var`. This replacement is effective because it removes the misleading lexical signal while keeping the program behavior, API calls, and data-flow relationships unchanged.

After the adversarial name is neutralized, the LLM no longer relies on the spurious benign prior introduced by the identifier. Its attention can return to the security-relevant evidence in the code: the potentially `NULL` return value is used before being checked. As a result, the downstream model recovers its vulnerability-detection capability and correctly predicts YES. This example demonstrates that HiPert does not need to repair the vulnerable program; it only removes adversarial prompt signals while preserving the code context needed for accurate vulnerability reasoning.

Chapter 5. Implementation

This chapter describes the concrete implementation and experimental configuration of HiPert. While the previous chapter defines the proposed defense methodology, this chapter explains how the defense system is instantiated in practice, what datasets and execution environment are used, how evaluation metrics are computed, and which baseline defenses are selected for comparison.

5.1 Experimental Setup

This section describes the experimental setup used to instantiate and evaluate HiPert. It consolidates the dataset, model configuration, and execution environment so that Chapter 6 can focus on parameter settings and evaluation results.

5.1.1 Datasets

The evaluation dataset contains paired benign and adversarial code samples for six indirect prompt injection attacks in code-generation scenarios: XOXO, ITGen, Flashboom, Shadow-Code, INSEC, and CoTDeceptor. Each data entry contains a benign code context and its corresponding adversarial version. The benign samples are used to measure false positives, while the adversarial samples are used to measure whether the defense can identify and sanitize malicious external code before it is provided to the downstream Code LLM.

These attacks cover three major categories of prompt-injection behavior. XOXO and ITGen

represent semantic trigger attacks, where malicious behavior is induced through natural-looking identifiers or context transformations. ShadowCode and INSEC represent adversarial perturbation attacks, where optimized abnormal token sequences are inserted into non-executable code regions. Flashboom represents decoy-based attacks that distract the model from the true security-relevant logic, and CoTDeceptor is used to evaluate attacks that manipulate the reasoning or generation trajectory of the model.

5.1.2 Models

We distinguish between the model used by the defense and the victim model protected by the defense. For the defense-side model, HiPert uses open-source code models to provide token likelihoods and logits. The evaluated defense-side models include Salesforce/codegen-350M-mono, Salesforce/codegen-350M-multi, Qwen/Qwen3.5-4B, and google/gemma-4-E4B. For victim models, we evaluate both open-source models and black-box coding agents, including CodeGemma, CodeBERT, GraphBERT, Gemma-family models, Mistral-family models, Claude-based coding agents, ChatGPT Codex-based coding agents, and Gemini-based coding agents.

This separation reflects the deployment setting discussed in the threat model. In a white-box setting, HiPert can use the victim model’s logits directly. In a black-box setting, the victim model does not expose logits, so HiPert uses an open-source surrogate model to detect and sanitize the retrieved code context before sending it to the black-box victim model.

5.1.3 Execution Environment

HiPert is implemented in Python as a preprocessing defense pipeline. The system supports C, C++, Java, Solidity, and Python through Tree-sitter parsers. For each supported language,

the implementation constructs a language-specific parser and extracts byte-level spans for comments, string literals, identifiers, parser-error regions, and decoy-like structures. Preserving byte offsets allows the system to edit only suspicious regions while keeping the remaining source code unchanged.

Model-based scoring is implemented with causal language models loaded through the Hugging Face Transformers framework. The defense model is configurable through the implementation interface, and the scripts use Salesforce/codegen-350M-mono as the default scoring model. The tokenizer is configured with left padding, and when no padding token is defined, the end-of-sequence token is used as the padding token. During inference, the model is executed in evaluation mode and uses reduced-precision computation when supported by the available device.

To reduce inference cost, candidate nodes are grouped by approximate token length and evaluated in microbatches. The implementation controls both the maximum number of rows in a batch and the maximum padded-token budget in a microbatch. This batching strategy prevents very long nodes from causing excessive padding overhead and makes runtime latency more stable across samples with different code lengths.

5.2 Implementation of Defense System

HiPert is implemented as a hierarchical preprocessing system placed between the retrieved external code context and the downstream Code LLM. Given an input code context, the system first normalizes the language label, selects the corresponding Tree-sitter parser and guardrail modules, and then applies three detection layers sequentially. The output of the pipeline records the layer-level detection decision and returns a sanitized version of the code when suspicious nodes are found.

Stage 1: syntax-guided pre-filtering. The first layer performs low-cost filtering over syntax-aware candidate regions. It uses regular-expression matching and structural anomaly checks over comments, string literals, identifiers, and parser-error regions. This layer detects explicit malicious patterns such as suspicious command strings, SQL-like payloads, path traversal patterns, prompt-template injection patterns, abnormal control characters, unusually long continuous strings, high non-ASCII ratios, and high special-character ratios. It also detects decoy-like structures by analyzing isolated or dead-looking function components. Since Stage 1 does not require model inference, it is used as the earliest and fastest filtering layer.

Stage 2: adversarial perturbation detection. The second layer targets adversarial perturbation attacks such as ShadowCode and INSEC. For each candidate node, the implementation computes token-level negative log-likelihood values using the defense language model. Instead of applying a fixed Min-K% ratio to every node, HiPert dynamically adjusts the selected token ratio according to node length and entropy. The layer then focuses on the most statistically surprising tokens and combines the resulting Min-K style score with local loss-spike information. Suspicious comments are removed, suspicious strings are replaced with empty string literals, and suspicious identifiers are replaced with neutral identifiers such as `tmp_var`.

Stage 3: semantic trigger detection. The third layer targets stealthy semantic triggers that may not look statistically abnormal. The system first selects high-risk candidate nodes from the syntax tree, including identifiers, strings, and comments. For each candidate, HiPert constructs neutralized variants and measures how much the model’s behavior changes after the candidate is replaced. A node is considered suspicious when its influence score or local z-score exceeds a dynamic threshold and its surprise score is sufficiently high. This design allows HiPert to detect natural-looking triggers that strongly affect the model’s generation behavior even when their surface form appears benign.

The three stages are executed sequentially in the full defense mode. Stage 1 removes obvious threats before expensive model-based scoring is applied. Stage 2 then handles statistically abnormal adversarial fragments. Stage 3 is applied to the remaining candidates and focuses on semantic influence. The implementation also supports layer-wise modes, allowing each layer to be evaluated independently for ablation analysis.

Thresholds are tuned before final evaluation. The tuning script extracts raw Stage 1 structural features, Stage 2 adversarial scores, and Stage 3 semantic influence features from paired benign and adversarial samples. The samples are split into training, validation, and test subsets, and the threshold search optimizes an F_β objective with $\beta = 1.5$ while penalizing high false-positive rates. After tuning, the selected threshold set is fixed and reused during final evaluation.

5.3 Evaluation Metrics

The evaluation treats adversarial samples as positive samples and benign samples as negative samples. A true positive (TP) is an adversarial sample correctly detected by the defense. A false negative (FN) is an adversarial sample missed by the defense. A true negative (TN) is a benign sample correctly accepted, and a false positive (FP) is a benign sample incorrectly flagged as malicious.

5.3.1 Precision, Recall, F1 Score, and FPR

Precision measures the reliability of positive predictions. A high precision indicates that most samples flagged by the defense are truly adversarial:

$$\text{Precision} = \frac{TP}{TP + FP}. \quad (5.1)$$

Recall measures the ability of the defense to detect adversarial samples. A high recall indicates that the defense misses fewer attacks:

$$\text{Recall} = \frac{TP}{TP + FN}. \quad (5.2)$$

The F1 Score summarizes the balance between precision and recall using their harmonic mean:

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (5.3)$$

False positive rate (FPR) measures how often benign samples are incorrectly classified as malicious:

$$\text{FPR} = \frac{FP}{FP + TN}. \quad (5.4)$$

In this work, F1 Score is used as the main overall detection metric, while FPR is separately reported because an overly aggressive defense may damage benign code-generation workflows even when its recall is high.

5.3.2 Detection Latency

Detection latency measures the runtime overhead introduced by HiPert before the downstream Code LLM receives the external code context. For each sample, latency is measured as the elapsed wall-clock time between the beginning and the end of the defense pipeline:

$$\text{Latency}_i = t_i^{\text{end}} - t_i^{\text{start}}. \quad (5.5)$$

The reported latency is the average latency over all evaluated benign and adversarial samples:

$$\text{Average Latency} = \frac{1}{N} \sum_{i=1}^N \text{Latency}_i. \quad (5.6)$$

Latency is reported in milliseconds. This metric is important because HiPert is designed as a preprocessing defense, so the detection process must remain practical for interactive code-

generation and coding-agent scenarios.

5.3.3 AUROC

Area Under the Receiver Operating Characteristic Curve (AUROC) evaluates the ranking quality of a detector independently of a single fixed threshold. For each threshold, the true positive rate and false positive rate are computed as follows:

$$\text{TPR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN}. \quad (5.7)$$

The ROC curve plots TPR against FPR under different threshold values. AUROC is then computed as the area under this curve:

$$\text{AUROC} = \int_0^1 \text{TPR}(\text{FPR}) d\text{FPR}. \quad (5.8)$$

A higher AUROC indicates that the detector assigns higher suspiciousness scores to adversarial samples than to benign samples more consistently. In the hierarchical setting, AUROC is computed from the raw detector scores before converting them into final binary decisions.

5.4 Baselines

We compare HiPert with three representative defense baselines.

KillBadCode detects suspicious tokens by measuring the cross-entropy change after removing each token from the input code. A token is considered suspicious if its removal causes a large naturalness change. This baseline represents token-level perturbation detection based on code naturalness.

CodeGarrison uses a hybrid learning-based architecture that combines syntax-aware and semantic representations, including Code2Vec, CodeBERT, and FastText-style token features.

This baseline represents learning-based poisoned-data detection.

DePA performs line-level perplexity analysis and sanitization. Instead of treating code as a flat word sequence, it computes line-level anomaly scores and removes lines whose perplexity deviates significantly from the file-level distribution. This baseline represents a code-oriented perplexity defense.



Chapter 6. Evaluation

This chapter evaluates whether HiPert can detect and sanitize indirect prompt injection attacks in code-generation systems. Building on the implementation details, experimental setup, metrics, and baselines defined in Chapter 5, we first describe the parameter settings used throughout the evaluation. We then organize the experimental results around four research questions: (1) whether HiPert outperforms existing state-of-the-art defenses, (2) whether the three-layer design is necessary, (3) whether the defense generalizes beyond the surrogate model used for detection and black-box victim models, and (4) whether HiPert remains robust against adaptive, transferred, and unseen attacks.

6.1 Parameter Setting

This section reports the parameter configuration used throughout the evaluation. The datasets, models, execution environment, evaluation metrics, and baseline definitions are described in Chapter 5.

For HiPert, the core parameters of each layer are selected through dynamic threshold tuning on the validation set. The objective is to maximize validation F1 Score while maintaining a low false positive rate on benign code.

For Layer 1, the pre-filter focuses on structural and statistical anomalies. The tuned parameters include the maximum token-length threshold, the non-ASCII character ratio threshold, and node-type-specific thresholds for strings, identifiers, comments, and syntax-error regions. The

default maximum token length is 100, and the default non-ASCII ratio threshold is 0.001.

For Layer 2, the adversarial guardrail uses CST-guided Dynamic Min-K% analysis. The main parameters are the universal adversarial threshold and the long-string-specific threshold. We set the default universal threshold to 10.0 and the long-string threshold to 15.0.

For Layer 3, the semantic guardrail uses CST-guided node perturbation analysis. The main parameters include the base influence threshold, the surprise tolerance, and the minimum surprise-score threshold. We set the default base influence threshold to 0.025, the surprise tolerance to 0.10, and the minimum surprise-score threshold to 0.15.

For the baselines, we follow their original settings when available and tune their thresholds on the validation set when the original paper does not specify a fixed configuration for our datasets. CodeGarrison is trained with Adam, a learning rate of 0.001, batch size 32, dropout 0.3, and early stopping. DePA uses Qwen3-4B-AWQ as the detection model with a maximum token length of 4096, and its anomaly threshold multiplier is tuned between 1.7 and 2.5. KillBadCode uses CodeBERT tokenization and n-gram cross-entropy following the original default statistical settings.

6.2 HiPert vs. SOTA Defenses

This section answers the main evaluation question: whether HiPert provides stronger detection than existing state-of-the-art defenses. We first report the main detection results across six attack families and then analyze threshold-agnostic performance using ROC curves.

6.2.1 Main Detection Result

Table 4 shows the overall detection results. HiPert achieves the best average F1 Score of 0.80, outperforming CodeGarrison with 0.70, DePA with 0.51, and KillBadCode with 0.30.

Table 4: Main Result

Attack Type	HiPert	CodeGarrison	DePA	KillBadCode
INSEC	0.90	0.73	0.61	0.32
Flashboom	0.79	0.85	0.55	0.28
XOXO	0.80	0.72	0.56	0.22
ShadowCode	0.85	0.62	0.54	0.32
CoTDeceptor	0.66	0.63	0.43	0.35
ITGen	0.77	0.65	0.34	0.30
Average	0.80	0.70	0.51	0.30

This improvement indicates that HiPert provides stronger detection than both learning-based and perplexity-based defenses under recent code-oriented prompt injection attacks.

The per-attack results further show why structure-aware and behavior-aware detection is necessary. HiPert obtains the best F1 Score on five of the six attack families, including INSEC, XOXO, ShadowCode, CoTDeceptor, and ITGen. Flashboom is the only case where CodeGarrison slightly exceeds HiPert, but HiPert still maintains strong performance and achieves the highest overall average. These results highlight the main contribution of HiPert: combining CST-guided code-structure analysis with model-behavior perturbation to detect both adversarial and semantic prompt-injection triggers.

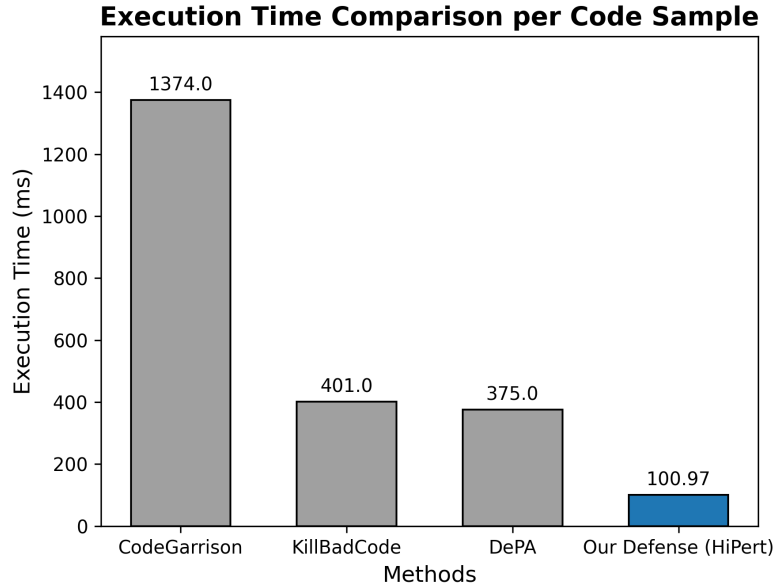


Figure 7: Execution time comparison per code sample.

Figure 7 compares the average execution time per code sample. HiPert requires only 100.97

ms, which is faster than DePA, KillBadCode, and CodeGarrison. This shows that HiPert improves detection effectiveness while keeping runtime overhead low.

6.2.2 Threshold-Agnostic Detection Performance

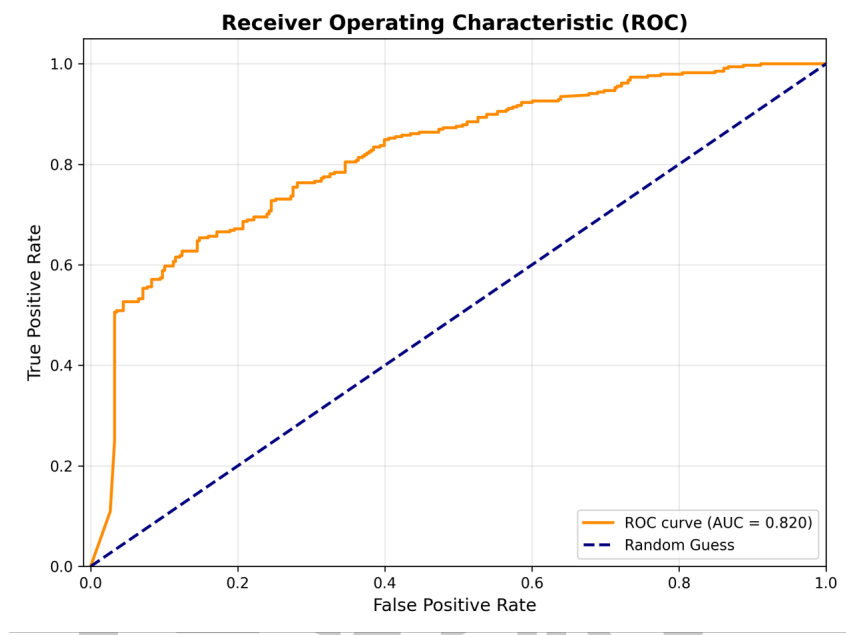


Figure 8: ROC curve.

Figure 8 evaluates whether HiPert assigns consistently higher threat scores to malicious prompts without relying on a single fixed threshold. The framework achieves an AUC of 0.82. This means that if one malicious prompt and one benign prompt are randomly selected, HiPert has an 82% probability of assigning a higher threat score to the malicious prompt.

The curve rises sharply near the lower-left region, where the false positive rate is close to zero. This indicates that HiPert can block a substantial portion of attacks while maintaining a low false positive rate. Therefore, the threat score produced by HiPert is meaningful beyond a single tuned threshold.

6.3 One Layer vs. Three Layers

This section evaluates whether the three-layer architecture is necessary. HiPert is not a single detector. It is a hierarchical defense in which each layer targets a different attack property: explicit or structural anomalies, adversarial likelihood spikes, and semantic influence over model generation.

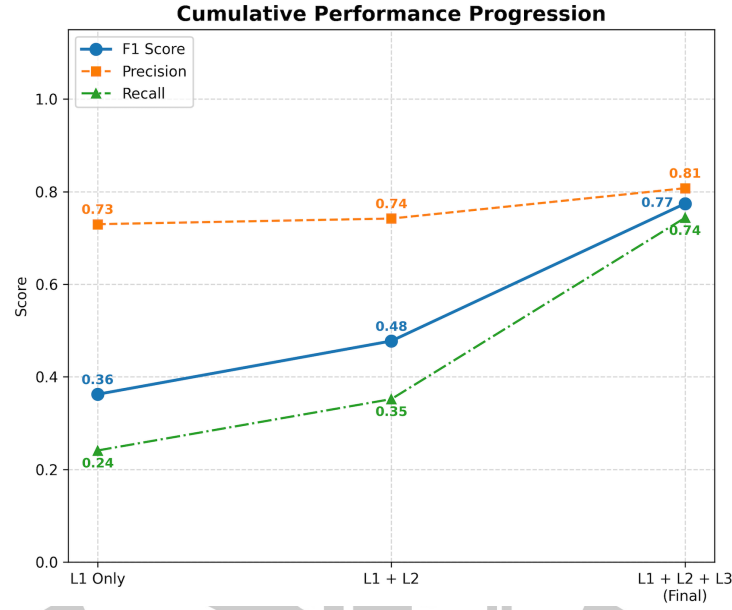


Figure 9: Cumulative layer performance.

6.3.1 Independent Contribution of Each Layer

Figure 9 shows that the three layers contribute complementary detection coverage rather than duplicating the same signal. Layer 1 handles low-cost structural filtering and explicit malicious patterns, including syntax-level anomalies and Flashboom-style decoy structures. Layer 2 captures adversarial likelihood spikes that commonly appear in ShadowCode and INSEC payloads. Layer 3 contributes the strongest semantic coverage because it directly measures how much a suspicious CST node changes the model’s output distribution after neutralization.

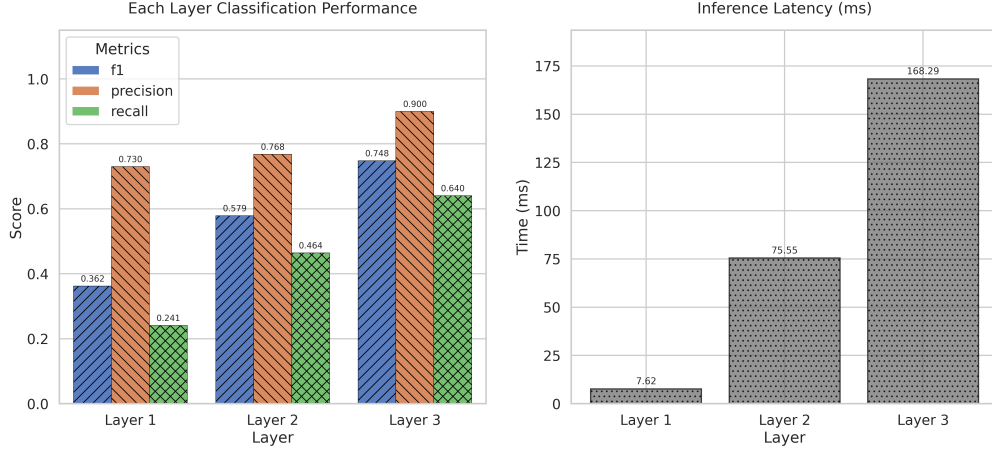


Figure 10: Each layer performance.

Figure 10 further isolates the behavior of each layer. Layer 3 has the strongest standalone detection ability, reaching a 64% interception ratio and an F1 Score of 0.748, but it also introduces the largest inference latency at 168.29 ms. This confirms that CST-guided node perturbation analysis is the most powerful component for difficult semantic attacks, but it is not suitable as the only always-on detector.

6.3.2 Cumulative Detection Performance

The cumulative results in Figure 9 show that the layers are complementary. As additional layers are enabled, the overall F1 Score and Recall increase while Precision remains stable, indicating that the hierarchical design improves detection coverage without introducing excessive false positives.

Combining all three layers also improves the latency-performance trade-off. Although Layer 3 alone is the strongest individual layer, the full pipeline reduces the average detection time to 104.64 ms while improving the F1 Score to 0.774. This occurs because Layer 1 and Layer 2 act as early-exit filters for simpler malicious inputs and also catch adversarial payloads that do not rely on semantic influence alone. Therefore, a single-stage defense is insufficient: static filtering misses natural-looking semantic triggers, perplexity-based filter-

ing misses low-perplexity attacks such as XOXO and ITGen, and perturbation-only detection spends unnecessary computation on cases that earlier layers can resolve more efficiently.

6.4 Surrogate Models vs. Black-Box Victim Models

This section evaluates whether HiPert generalizes beyond a single logits provider and whether it remains effective when the protected victim model is black-box. This is important because HiPert does not require logits from the final victim model. In black-box deployment, HiPert can use an open-source surrogate model for detection and sanitization before forwarding the sanitized context to the victim model.

6.4.1 Cross-Surrogate Generalization

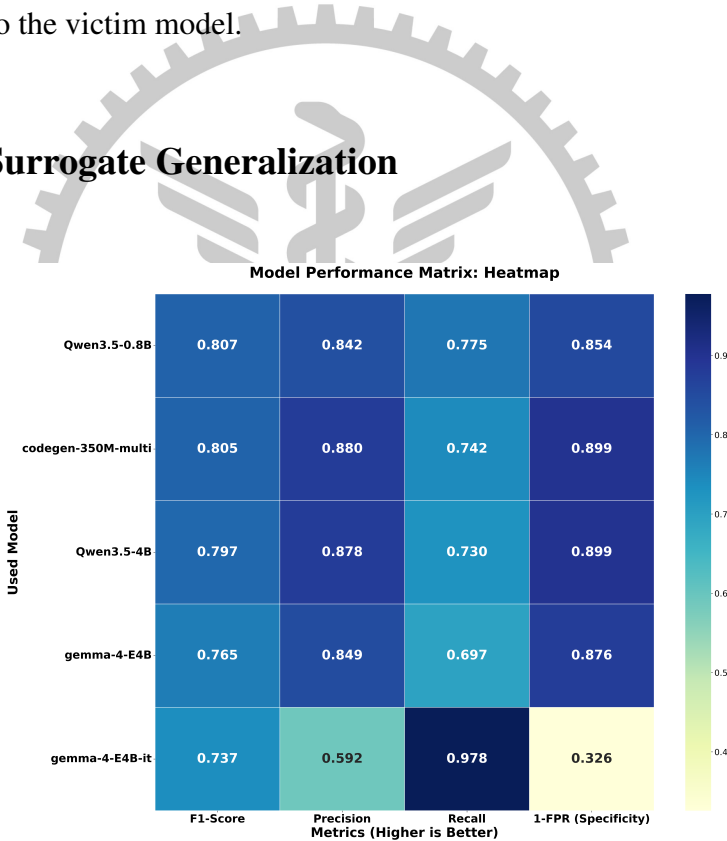


Figure 11: Cross model generalization.

Figure 11 evaluates whether HiPert overfits to a single defense-side model. When different open-source models are used as the logits provider, the F1 Score remains between 0.737 and 0.807 across the evaluated attacks. This indicates that the proposed detection signals are not tied to a single surrogate model.

This result is especially important for black-box victim model settings. Since the victim model may not expose logits, the defense must rely on a locally deployed surrogate model. The stable performance across multiple surrogate models suggests that HiPert captures attack-induced behavior changes rather than model-specific artifacts.

6.4.2 Attack Mitigation on Open-Weight Victim Models

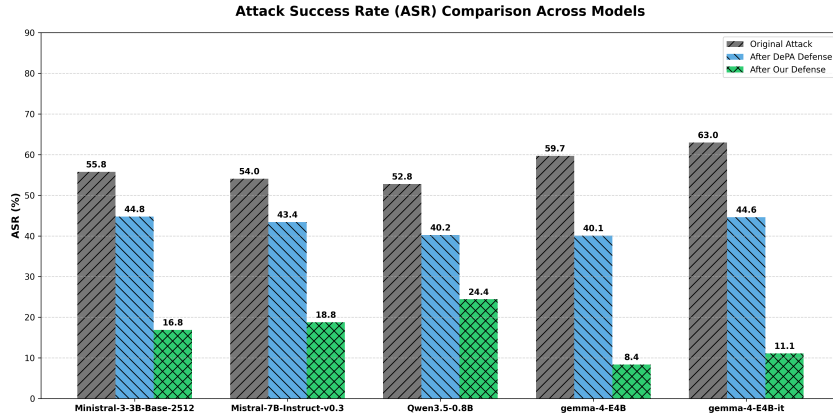


Figure 12: ASR on different victim models.

Detection is useful only if it reduces the attack effect on downstream victim models. In this experiment, the attacks are generated from the six prompt injection datasets, the defense model is Salesforce/codegen-350M-multi, and the victim models include Ministral-3-3B-Base-2512, Mistral-7B-Instruct-v0.3, Qwen3.5-0.8B, gemma-4-E4B, and gemma-4-E4B-it.

As shown in Figure 12, the original ASR of 64%–75% is reduced to 10.5%–21.5% after detection and sanitization. Across Ministral, Mistral, Qwen, and Gemma models, HiPert reduces ASR more effectively than DePA by approximately 20%–35%. This indicates that HiPert removes model-facing triggers rather than merely labeling the input as malicious.

Table 5: Evaluation of commercial black-box API models

Victim Model	Before Defense (ASR)	After Defense (ASR)	Impact
claude-3.5-haiku	27.11%	7.72%	-19.39%
chatgpt-5.1-codex-mini	18.32%	5.81%	-12.51%
gemini-3.1-flash-lite	24.14%	8.28%	-15.86%

6.4.3 Evaluation of commercial black-box API models

Table 5 reports black-box testing results on coding agents accessed through APIs. HiPert reduces ASR from 27.11% to 7.72% on claude-3.5-haiku, from 18.32% to 5.81% on chatgpt-5.1-codex-mini, and from 24.14% to 8.28% on gemini-3.1-flash-lite. These results show that the sanitization effect transfers to commercial black-box API models, even when the defense cannot inspect the victim model internals.

6.4.4 Impact on Clean-Code Utility

Table 6: Clean-code utility.

Benchmark	Metric	Before Defense	After Defense	Impact
HumanEval	Pass@1 (%)	100.00	100.00	0.00
HumanEval	Compile (%)	100.00	100.00	0.00
MBPP	Pass@1 (%)	96.11	89.88	-6.23
MBPP	Compile (%)	100.00	100.00	0.00
RepoBench	Exact Match (%)	100.00	67.33	-32.67
RepoBench	BLEU-4 Score	100.00	94.05	-5.95

Table 6 reports the impact of HiPert on benign code-generation utility. HiPert preserves compile rates on HumanEval and MBPP, showing that the sanitization process does not break syntactic correctness. The MBPP Pass@1 score drops by 6.23 percentage points, while RepoBench shows a larger exact-match decrease but a smaller BLEU-4 decrease. This suggests that HiPert may alter surface-level code tokens during sanitization while largely preserving compilability and semantic similarity.

6.5 Known Attacks vs. Adaptive and Unseen Attacks

The previous sections evaluate HiPert under standard attack settings. However, a practical defense must also remain effective when attackers are aware of the detection mechanism, when attacks transfer across models, and when the defense faces unseen attack families. Therefore, we conduct three robustness analyses: adaptive attacks, transfer attacks, and leave-one-attack-out evaluation.

6.5.1 Adaptive Attack

Adaptive attacks represent a stronger threat model in which the attacker knows the defense architecture and detection algorithms. We evaluate three adaptive strategies designed to target different components of HiPert.

- **Decoy Injection.** The attacker injects high-surprise but non-malicious nodes to occupy candidate slots in Layer 3, attempting to prevent the real malicious trigger from being evaluated.
- **Copy Trigger.** The attacker injects multiple identical malicious triggers in different locations. If only one trigger is neutralized, the remaining triggers may preserve the malicious behavior and reduce the influence change of any single node.
- **Contextual Attack.** The attacker uses a genetic algorithm to optimize the surrounding context of the malicious payload and reduce its surprise score, making the trigger harder to select for perturbation analysis.

As shown in Table 7, the F1 Score decreases from 0.82 to 0.74 under Decoy Injection, 0.62 under Copy Trigger, and 0.66 under Contextual Attack. Copy Trigger causes the largest

Table 7: Original attacks vs. adaptive attacks

Attack Type	Original Defense	Adaptive Attack	Impact
Decoy Injection	0.82	0.74	-0.08
Copy Trigger	0.82	0.62	-0.20
Contextual Attack	0.82	0.66	-0.16

degradation because duplicated triggers dilute the influence of any single neutralized node. Nevertheless, HiPert still maintains an F1 Score of at least 0.62, which remains substantially higher than KillBadCode’s baseline F1 Score of 0.288.

6.5.2 Transfer Attack

Transfer attacks evaluate whether malicious payloads generated under one optimization setting can still be detected when they are evaluated under a different defense or victim setting. This experiment is important because black-box attackers often optimize payloads with an accessible surrogate model, tokenizer, or baseline defense, and then transfer the optimized payloads to the protected system.

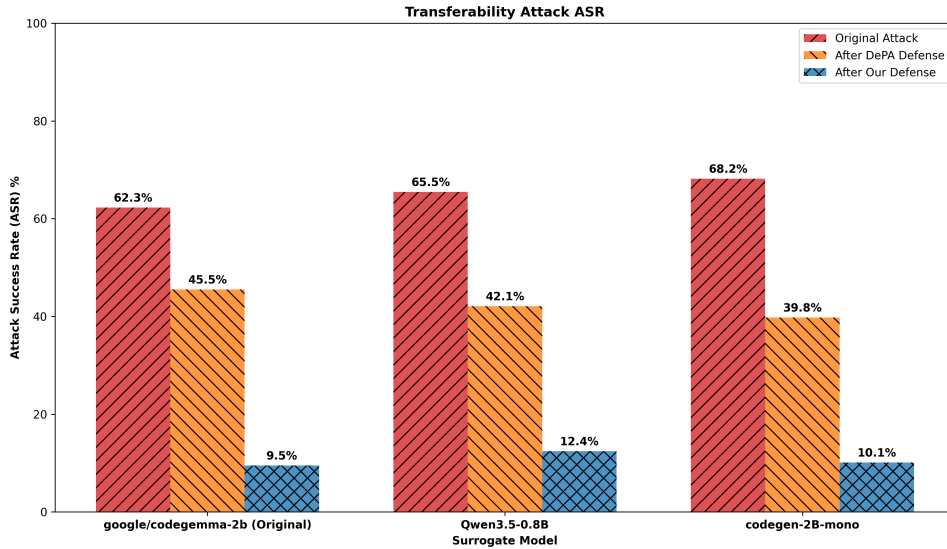


Figure 13: Transferability effect on ASR.

Figure 13 evaluates transferability attack ASR. In the cross-model transfer setting, INSEC and ShadowCode payloads are generated with surrogate models such as google/codegemma-2b,

Qwen3.5-0.8B, and codegen-2B-mono, and then evaluated on the victim model gemma-4-E4B-it. Without defense, these transferred payloads achieve high ASR, showing strong transferability. After applying HiPert, ASR is reduced from 62%–68% to approximately 10%–12%, demonstrating that HiPert can remove transferred malicious triggers before they reach the victim model.

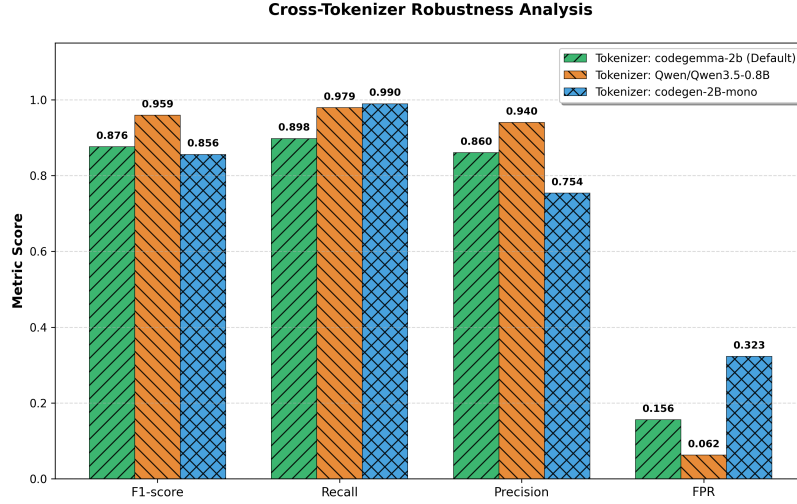


Figure 14: Cross-tokenizer robustness.

Figure 14 evaluates tokenization robustness. This setting tests whether HiPert overfits to a specific tokenization scheme. This is a critical property because INSEC and Shadow-Code depend heavily on the token space used during payload optimization. For example, if the attacker optimizes a nonsensical trigger with Qwen’s tokenizer, while the defense uses Salesforce/codegen-350M logits and tokenizer for detection, the payload is represented under a different token space at detection time. Strong performance in this setting indicates that HiPert is not merely memorizing tokenizer-specific artifacts. Instead, it captures abnormal likelihood patterns and node-level influence that remain detectable even when the attack and defense use different tokenizers.

Table 8 evaluates a baseline-targeted transfer attack. In this setting, the attacker explicitly optimizes against DePA using a genetic algorithm. The search process selects token combi-

Table 8: Defense Against DePA-Targeted Attacks

Defense	Original F1	Targeted Attack F1	Impact
HiPert	0.8061	0.7202	0.0859
DePA	0.5041	0.2300	0.2741

nations from the vocabulary that produce very low line-level perplexity, and then uses those combinations as natural-looking comment triggers. This design directly targets DePA’s line-level perplexity assumption: by making the trigger appear statistically natural at the line level, the attacker can substantially reduce DePA’s detection ability.

HiPert remains more robust under this baseline-targeted attack because it does not rely only on line-level perplexity. Even when the trigger has low line-level PPL, the CST-guided pipeline can still evaluate the suspicious comment node and measure whether neutralizing that node causes a large change in the model’s output distribution. Therefore, the DePA-targeted experiment shows two points: first, a baseline defense can be weakened when the attacker optimizes specifically against its detection signal; second, HiPert provides stronger protection because it combines structural localization, token-level anomaly analysis, and node-level semantic influence.

6.5.3 Leave-One-Attack-Out Evaluation

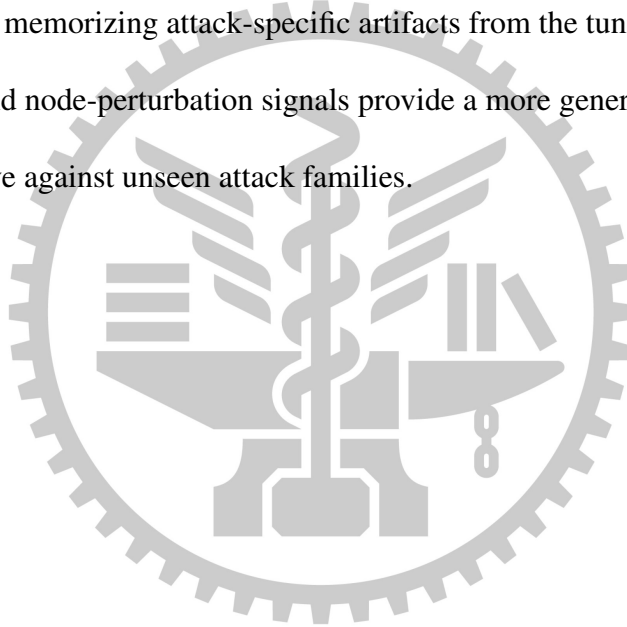
Leave-one-attack-out evaluation tests whether HiPert generalizes to unseen attack families rather than overfitting to attack-specific artifacts during threshold tuning. In each round, one attack family is excluded during grid-search-based hyperparameter tuning and is used only for testing. This setting simulates a practical deployment scenario in which a new attack family appears after the defense has already been configured.

Table 9 shows that HiPert maintains stable performance even when each attack family is excluded from the hyperparameter tuning stage. The average F1 Score decreases only from

Table 9: Leave-one-attack-out evaluation results.

Attack Type	Original Defense	Leave-One-Attack-Out	Impact
XOXO	0.80	0.673	-0.127
ITGen	0.77	0.63	-0.14
Flashboom	0.79	0.865	0.075
CoTDeceptor	0.66	0.66	0.00
ShadowCode	0.85	0.85	0.00
INSEC	0.90	0.90	0.00
Average	0.80	0.763	-0.037

0.80 to 0.763, corresponding to a small absolute drop of 0.037. CoTDeceptor, ShadowCode, and INSEC show no degradation under the leave-one-attack-out setting, while Flashboom improves from 0.79 to 0.865 when it is held out from tuning. These results indicate that HiPert’s detection logic is not merely memorizing attack-specific artifacts from the tuning data. Instead, the CST-guided anomaly and node-perturbation signals provide a more general defense mechanism that can remain effective against unseen attack families.



Chapter 7. Conclusion

This thesis presents HiPert, a hierarchical syntax-semantic defense framework for mitigating indirect prompt injection attacks against Code LLMs. The work studies attacks hidden in comments, strings, identifiers, and decoy code, and combines static code analysis with model-behavior analysis to detect and sanitize malicious triggers before they reach a downstream code model.

The main conclusions are as follows.

- **Syntax-aware localization is useful for code defense.** HiPert uses Tree-sitter to identify high-risk code nodes and preserve byte-level spans for sanitization. This allows the system to inspect model-facing regions without rewriting the entire program.
- **Different attack styles require different signals.** Explicit or structural attacks can be handled by pre-filtering, adversarial perturbations can be detected through localized loss patterns, and natural-looking semantic triggers require perturbation-based influence analysis.
- **Sanitization reduces downstream attack success.** Experiments on six prompt injection attacks show that HiPert achieves an average F1 Score of 0.80. After sanitization, the ASR of victim models is reduced from 64%–75% to 10.5%–21.5%.
- **The framework generalizes across settings.** HiPert maintains stable performance when different open-source models are used for detection. It also remains effective under adaptive attacks, transfer-based attacks, and tokenizer changes.

Overall, this thesis shows that code-specific structure and model-behavior sensitivity can be combined into an effective defense for Code LLM-based generation systems. Future work can extend HiPert to larger repository-scale contexts, richer programming languages, and interactive agent workflows where the model can read, edit, and execute files across multiple turns.



References

- [1] E. Ghannoum and M. Ghafari, “Poisoned Source Code Detection in Code Models,” Feb. 2025.
- [2] C.-C. Tsai, C.-M. Yu, Y.-D. Lin, Y.-S. Wu, and W.-B. Lee, “Beyond Natural Language Perplexity: Detecting Dead Code Poisoning in Code Generation Datasets,” Feb. 2025, arXiv:2502.20246 [cs].
- [3] W. Sun, Y. Chen, M. Yuan, C. Fang, Z. Chen, C. Wang, Y. Liu, B. Xu, and Z. Chen, “Show Me Your Code! Kill Code Poisoning: A Lightweight Method Based on Code Naturalness,” Feb. 2025, arXiv:2502.15830 [cs].
- [4] X. Li, H. Li, Yue Wu, Y. Zhang, K. Xu, X. Cheng, S. Zhong, and F. Xu, “Make a Feint to the East While Attacking in the West: Blinding LLM-Based Code Auditors with Flash-boom Attacks.”
- [5] L. Huang, W. Sun, and M. Yan, “Iterative Generation of Adversarial Example for Deep Code Models,” in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, Apr. 2025, pp. 2213–2224, iSSN: 1558-1225.
- [6] A. torek, M. Gupta, N. Bhatt, A. Gupta, J. Kim, P. Srivastava, and S. Jana, “XOXO: Stealthy Cross-Origin Context Poisoning Attacks against AI Coding Assistants,” May 2025, arXiv:2503.14281 [cs].

- [7] S. Jenko, N. Mündler, J. He, M. Vero, and M. Vechev, “Black-Box Adversarial Attacks on LLM-Based Code Completion,” Jun. 2025, arXiv:2408.02509 [cs].
- [8] “Your AI, My Shell.”
- [9] M. Wahed, X. Zhou, K. A. Nguyen, T. Yu, N. Diwan, G. Wang, D. Hakkani-Tür, and I. Lourentzou, “MOCHA: Are Code Language Models Robust Against Multi-Turn Malicious Coding Prompts?” Jul. 2025, arXiv:2507.19598 [cs].
- [10] Y. Yang, Y. Li, H. Yao, B. Yang, Y. He, T. Zhang, D. Tao, and Z. Qin, “ShadowCode: Towards (Automatic) External Prompt Injection Attack against Code LLMs,” Jul. 2025, arXiv:2407.09164 [cs].
- [11] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, “Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models,” in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1*, Jul. 2025, pp. 1809–1820, arXiv:2312.14197 [cs].
- [12] T. Wen, C. Wang, X. Yang, H. Tang, and w. k. i. a. p. l. t. Xie, Yueqi ... [omitted authors for line length constraints, “Defending against Indirect Prompt Injection by Instruction Detection,” in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 19 472–19 487, arXiv:2505.06311 [cs].
- [13] Y. Chen, H. Li, Y. Sui, Y. He, Y. Liu, Y. Song, and B. Hooi, “Can Indirect Prompt Injection Attacks Be Detected and Removed?” Oct. 2025, arXiv:2502.16580 [cs].
- [14] M. Liu, L. Jiang, Y. Liang, S. S. Du, Y. Choi, T. Althoff, and N. Jaques, “Chasing Moving Targets with Online Self-Play Reinforcement Learning for Safer Language Models,” Oct. 2025, arXiv:2506.07468 [cs].

- [15] H. Lin, Y. Lao, T. Geng, T. Yu, and W. Zhao, “UniGuardian: A Unified Defense for Detecting Prompt Injection, Backdoor Attacks and Adversarial Attacks in Large Language Models,” Feb. 2025, arXiv:2502.13141 [cs].
- [16] S. Ouyang, Y. Qin, B. Lin, L. Chen, X. Mao, and S. Wang, “Smoke and Mirrors: Jailbreaking LLM-based Code Generation via Implicit Malicious Prompts,” Mar. 2025, arXiv:2503.17953 [cs].
- [17] Y. Jiao, X. Wang, and K. Yang, “PR-Attack: Coordinated Prompt-RAG Attacks on Retrieval-Augmented Generation in Large Language Models via Bilevel Optimization,” Jun. 2025, arXiv:2504.07717 [cs].
- [18] M. Liu, L. Jiang, Y. Liang, S. S. Du, Y. Choi, T. Althoff, and N. Jaques, “Chasing Moving Targets with Online Self-Play Reinforcement Learning for Safer Language Models,” Oct. 2025, arXiv:2506.07468 [cs].
- [19] H. Li, H. Gao, Z. Zhao, Z. Lin, J. Gao, and X. Li, “LLMs Caught in the Crossfire: Malware Requests and Jailbreak Challenges,” Jun. 2025, arXiv:2506.10022 [cs].
- [20] H. Li, M. Li, J. Zuo, S. Li, X. Li, H. Wu, Y. Lu, and X. He, “CoTDeceptor: Adversarial Code Obfuscation Against CoT-Enhanced LLM Code Agents,” 2025, arXiv:2512.21250 [cs].
- [21] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, D. Wagner, and C. Guo, “Se-cAlign: Defending Against Prompt Injection with Preference Optimization,” in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, 2025, arXiv:2410.05451 [cs].

- [22] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, “StruQ: Defending Against Prompt Injection with Structured Queries,” in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, arXiv:2402.06363 [cs].
- [23] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, “Defeating Prompt Injections by Design,” Mar. 2025, arXiv:2503.18813 [cs].